

Documentacion del Proyecto Figuras Que Ensenan

Equipo de desarrollo

2026-05-21

Table of Contents

1. Introduccion	1
1.1. Publico Objetivo	2
1.2. Enfoque del Proyecto	2
2. Requisitos Globales del Sistema	3
2.1. Requisitos Funcionales Globales	3
2.2. Requisitos No Funcionales Globales	4
2.3. Reglas de Negocio Globales	5
3. Casos de Uso	7
3.1. Actores del Sistema	7
3.2. Diagramas de Casos de Uso	7
3.2.1. Estudiante	7
3.2.2. Profesor, Tutor o Padre	9
3.2.3. Administrador	11
3.3. Casos de Uso del Estudiante	13
3.4. Casos de Uso del Profesor, Tutor o Padre	13
3.5. Casos de Uso del Administrador	14
3.6. Descripcion Detallada de Casos de Uso Principales	15
3.6.1. CU Principal 01: Iniciar sesion	15
3.6.2. CU Principal 02: Registrar usuario	16
3.6.3. CU Principal 03: Consultar progreso del estudiante	17
3.6.4. CU Principal 04: Jugar nivel	18
3.6.5. CU Principal 05: Consultar logs y monitorear servicios	19
4. Microservicios del Sistema	21
4.1. API Gateway	21
4.2. Microservicio de Autenticacion	22
4.3. Microservicio de Datos	23
4.4. Servicio de Deteccion de Manos	26
4.5. Microservicio de Logs	27
4.6. Listener RabbitMQ-MongoDB	28
5. Clientes del Sistema	29
5.1. Cliente Web	29
5.2. Cliente WPF Administrativo	30
5.3. Cliente Android	31
5.4. Justificacion General de los Clientes	32
6. Diagrama de Componentes	33
6.1. Justificacion de la Arquitectura de Componentes	33
7. Diagrama de Despliegue Docker	35
7.1. Puertos Publicados por Contenedor	35

7.2. Diagrama de Despliegue	35
7.3. Acceso a Servicios	36
8. Persistencia de Datos	38
8.1. Base de Datos Relacional: PostgreSQL	38
8.1.1. Funcion Dentro del Sistema	38
8.1.2. Diagrama PlantUML de la Base de Datos PostgreSQL	39
8.1.3. Justificacion del Modelo Relacional	40
8.2. Base de Datos No Relacional: MongoDB	40
8.2.1. Funcion Dentro del Sistema	41
8.2.2. Ejemplo de Documento de Log	41
8.2.3. Estructura Detectada en MongoDB	41
8.2.4. Diagrama PlantUML de MongoDB	42
8.2.5. Justificacion del Modelo No Relacional	43
9. RabbitMQ	44
9.1. Montaje en Docker	44
9.2. Cola Principal del Proyecto	44
9.3. Flujo General	45
9.4. Diagrama del Flujo de Mensajeria	45
9.5. Productor de Mensajes	45
9.6. Criterios para Publicar Logs	46
9.7. Estructura del Mensaje	46
9.8. Consumidor de Mensajes	47
9.9. Confirmacion y Reintentos	47
9.10. Metadatos Guardados	48
9.11. Cola Utilizada por el Proyecto	48
9.12. Consideraciones	48
10. JWT, Autenticacion y Autorizacion por Roles	49
10.1. Generacion del JWT en Authentication Service	49
10.2. Informacion Incluida en el Token	49
10.3. Uso del JWT en los Clientes	50
10.4. Papel del API Gateway	50
10.5. Validacion y Autorizacion en el Microservicio de Datos	50
10.6. Determinacion del Rol	51
11. Pruebas Automatizadas con Newman	52
12. Pruebas de Estres con JMeter	56

Chapter 1. Introduccion

Figuras Que Ensenan es una aplicacion educativa interactiva disenada para que ninas y ninos, especialmente en edades tempranas, aprendan conceptos matematicos de forma natural, intuitiva y divertida. Su interes principal es transformar el aprendizaje en una experiencia activa: el estudiante no solo ve ejercicios en pantalla, sino que participa mediante el movimiento, la interaccion y la resolucion de retos dentro de una narrativa ludica.

El proyecto combina recursos pedagogicos, tecnologia e interaccion digital para apoyar el aprendizaje de la suma. Una de sus ideas centrales es permitir que el nino utilice sus dedos como parte del proceso de aprendizaje, acercando la actividad a una experiencia concreta y corporal. Esta propuesta se relaciona con el enfoque CPA del metodo Singapur, donde el estudiante primero manipula o representa elementos concretos, despues interpreta una representacion visual y finalmente comprende la relacion abstracta entre cantidades y operaciones.

Desde la perspectiva del estudiante, la aplicacion busca que aprender no se sienta como una actividad rigida o repetitiva. La narrativa, los niveles, las sesiones de uso y los resultados permiten construir una experiencia mas cercana al juego, donde cada avance puede registrarse y convertirse en informacion util para dar seguimiento a su progreso. El objetivo no es solamente resolver sumas, sino favorecer la motivacion, la confianza y la participacion activa del nino durante su proceso de aprendizaje.

Sin embargo, Figuras Que Ensenan no esta pensada unicamente como una aplicacion para ninos. Tambien incorpora funciones para las personas que acompanian su desarrollo. Los tutores, profesores o padres pueden consultar informacion de los estudiantes asociados a su cuenta, revisar sesiones registradas, analizar resultados por nivel y dar seguimiento al avance de varios ninos desde una misma plataforma. Esto permite que el acompanamiento no dependa solo de la supervision directa, sino tambien de datos generados durante el uso real de la aplicacion.

El sistema tambien considera un perfil administrativo. El administrador cuenta con herramientas para obtener estadisticas generales sobre estudiantes, tutores, registros, actividad, horas jugadas y niveles superados. Ademias, el cliente administrativo permite consultar logs y monitorear servicios, lo que ayuda a tener una vision mas completa tanto del uso educativo como del estado tecnico de la plataforma.

A nivel tecnico, el proyecto esta organizado como un ecosistema de aplicaciones y servicios. Incluye clientes web, Android y WPF; un servicio de autentificacion con roles para estudiantes, tutores y administradores; un gateway para centralizar la comunicacion; microservicios para gestionar estudiantes, tutores, sesiones, niveles y resultados; y componentes de mensajeria y almacenamiento de logs mediante RabbitMQ y MongoDB. Esta arquitectura permite separar responsabilidades, controlar el acceso a la informacion y facilitar el crecimiento del sistema.

En conjunto, Figuras Que Ensenan busca resolver la necesidad de una herramienta educativa que no solo ayude al nino a aprender de manera mas visual, interactiva y significativa, sino que tambien ofrezca a profesores, padres y administradores informacion clara para acompanarlo, evaluar su progreso y tomar mejores decisiones sobre su proceso de aprendizaje.

1.1. Publico Objetivo

El proyecto esta dirigido principalmente a:

- Ninas y ninos que utilizan la aplicacion para practicar y reforzar aprendizajes matematicos.
- Profesores o tutores que acompanian el avance de uno o varios estudiantes.
- Padres o responsables que desean consultar el progreso de sus hijos.
- Administradores que supervisan estadisticas, actividad, logs y estado general de los servicios.

1.2. Enfoque del Proyecto

La propuesta de Figuras Que Ensenan se apoya en tres ideas principales:

- Aprendizaje interactivo, usando actividades visuales, movimiento y retroalimentacion.
- Seguimiento educativo, registrando sesiones, niveles y resultados para consultar el progreso.
- Administracion y monitoreo, permitiendo analizar estadisticas y revisar el comportamiento del sistema.

Chapter 2. Requisitos Globales del Sistema

Esta seccion define los requisitos generales de Figuras Que Ensenan como sistema completo. No describe todavia los requisitos particulares de cada microservicio, sino las capacidades y cualidades que debe cumplir la plataforma considerando todos sus componentes: clientes, gateway, servicios backend, deteccion de manos, bases de datos, mensajeria, logs y modulo administrativo.

2.1. Requisitos Funcionales Globales

ID	Requisito	Prioridad
RF-G-01	El sistema debe permitir el registro de usuarios con rol de estudiante o tutor, solicitando los datos necesarios segun el tipo de usuario.	Alta
RF-G-02	El sistema debe permitir el inicio de sesion de estudiantes, tutores y administradores mediante credenciales validas.	Alta
RF-G-03	El sistema debe generar y utilizar tokens de autenticacion para identificar el usuario activo y su rol dentro de la plataforma.	Alta
RF-G-04	El sistema debe dirigir a cada usuario hacia las funcionalidades correspondientes a su rol: estudiante, tutor o administrador.	Alta
RF-G-05	El sistema debe permitir que un estudiante consulte su informacion personal, su tutor asignado, sus sesiones y sus resultados de actividad.	Alta
RF-G-06	El sistema debe permitir que un tutor consulte su perfil y la lista de estudiantes asociados a su cuenta.	Alta
RF-G-07	El sistema debe permitir que un tutor asigne o vincule estudiantes a su cuenta mediante informacion de identificacion, como el correo del estudiante.	Alta
RF-G-08	El sistema debe permitir que un tutor consulte el detalle de cada estudiante asignado, incluyendo informacion personal, sesiones registradas y resultados obtenidos.	Alta
RF-G-09	El sistema debe registrar sesiones de uso asociadas a estudiantes, incluyendo datos como fecha de inicio, fecha de termino y dispositivo utilizado.	Alta
RF-G-10	El sistema debe registrar resultados de niveles o actividades, incluyendo intentos, fallos, tiempo de finalizacion y estado de completado.	Alta
RF-G-11	El sistema debe permitir consultar niveles, sesiones y resultados para dar seguimiento al progreso educativo de los estudiantes.	Media

ID	Requisito	Prioridad
RF-G-12	El sistema debe permitir que el administrador inicie sesion en un cliente administrativo independiente.	Alta
RF-G-13	El sistema debe permitir que el administrador consulte estadisticas generales sobre estudiantes, tutores, administradores, registros, actividad, horas jugadas y niveles superados.	Alta
RF-G-15	El sistema debe exponer un gateway como punto de entrada para centralizar la comunicacion entre clientes y servicios internos.	Alta
RF-G-16	El sistema debe redirigir solicitudes de autentificacion, datos, logs, salud y otros modulos hacia los servicios correspondientes.	Alta
RF-G-17	El sistema debe publicar y almacenar eventos o logs relevantes generados por los servicios.	Media
RF-G-18	El sistema debe permitir consultar logs del sistema para revisar actividad, errores o eventos importantes.	Media
RF-G-19	El sistema debe permitir el monitoreo basico del estado de los servicios mediante endpoints de salud.	Media
RF-G-20	El sistema debe permitir el acceso desde diferentes clientes, incluyendo aplicacion web, aplicacion Android y cliente administrativo WPF.	Alta
RF-G-21	El sistema debe permitir enviar imagenes al Gateway para obtener el conteo de dedos mediante el servicio de deteccion de manos.	Alta

2.2. Requisitos No Funcionales Globales

ID	Requisito	Prioridad
RNF-G-01	El sistema debe proteger el acceso a la informacion mediante autentificacion y autorizacion basada en roles.	Alta
RNF-G-02	El sistema debe garantizar que los estudiantes solo accedan a su propia informacion y que los tutores solo accedan a estudiantes asociados a su cuenta.	Alta
RNF-G-03	El sistema debe restringir las funciones administrativas a usuarios con rol de administrador.	Alta
RNF-G-04	El sistema debe manejar contraseñas de forma segura, evitando almacenar contraseñas en texto plano.	Alta
RNF-G-05	El sistema debe mantener una arquitectura modular basada en clientes, gateway, servicios backend, bases de datos y componentes de mensajería.	Alta

ID	Requisito	Prioridad
RNF-G-06	El sistema debe permitir que sus componentes se ejecuten de forma independiente para facilitar mantenimiento, pruebas y despliegue.	Media
RNF-G-07	El sistema debe ofrecer una interfaz clara, sencilla y accesible para niños con habilidades lectoras en desarrollo.	Alta
RNF-G-08	El sistema debe evitar interfaces saturadas para el estudiante y favorecer estímulos visuales claros, retroalimentación positiva e interacciones comprensibles.	Alta
RNF-G-09	El sistema debe responder de forma adecuada ante errores, devolviendo mensajes comprensibles y estados HTTP consistentes en sus APIs.	Media
RNF-G-10	El sistema debe registrar eventos relevantes para facilitar auditoría, monitoreo y análisis técnico.	Media
RNF-G-11	El sistema debe permitir consultar el estado de servicios críticos para detectar fallas o indisponibilidad.	Media
RNF-G-12	El sistema debe usar variables de entorno o archivos de configuración para manejar datos sensibles como cadenas de conexión, secretos JWT y URLs de servicios.	Alta
RNF-G-13	El sistema no debe documentar ni exponer credenciales reales en archivos públicos de documentación.	Alta
RNF-G-14	El sistema debe permitir persistencia de datos educativos, usuarios, sesiones y resultados en una base de datos principal.	Alta
RNF-G-15	El sistema debe permitir persistencia de logs y eventos en una base de datos documental cuando corresponda.	Media
RNF-G-16	El sistema debe poder ejecutarse en un entorno local de desarrollo con dependencias como Node.js, .NET, MongoDB, RabbitMQ y base de datos relacional.	Media
RNF-G-17	El sistema debe facilitar su crecimiento futuro, permitiendo agregar nuevos niveles, nuevas métricas, nuevos clientes o nuevos servicios.	Media
RNF-G-18	El sistema debe mantener separadas las responsabilidades entre autenticación, datos del dominio, gateway, detección de abusos, logs y clientes.	Alta

2.3. Reglas de Negocio Globales

- El estudiante solo puede consultar y modificar información asociada a su propio identificador.
- El tutor solo puede consultar información de estudiantes que estén asociados a su cuenta.

- El tutor solo puede consultar sesiones y resultados de estudiantes asignados.
- Las funciones administrativas requieren rol de administrador.
- El acceso a datos se controla mediante tokens JWT y validacion de rol.

Chapter 3. Casos de Uso

Esta seccion describe los casos de uso globales identificados a partir de los clientes y servicios del proyecto. Para efectos de la documentacion, el rol de profesor se considera dentro del perfil **tutor**, ya que en el sistema este usuario representa a la persona adulta que acompaña, supervisa o da seguimiento a uno o varios estudiantes. Este rol tambien puede corresponder a padres o responsables del acompanamiento educativo.

3.1. Actores del Sistema

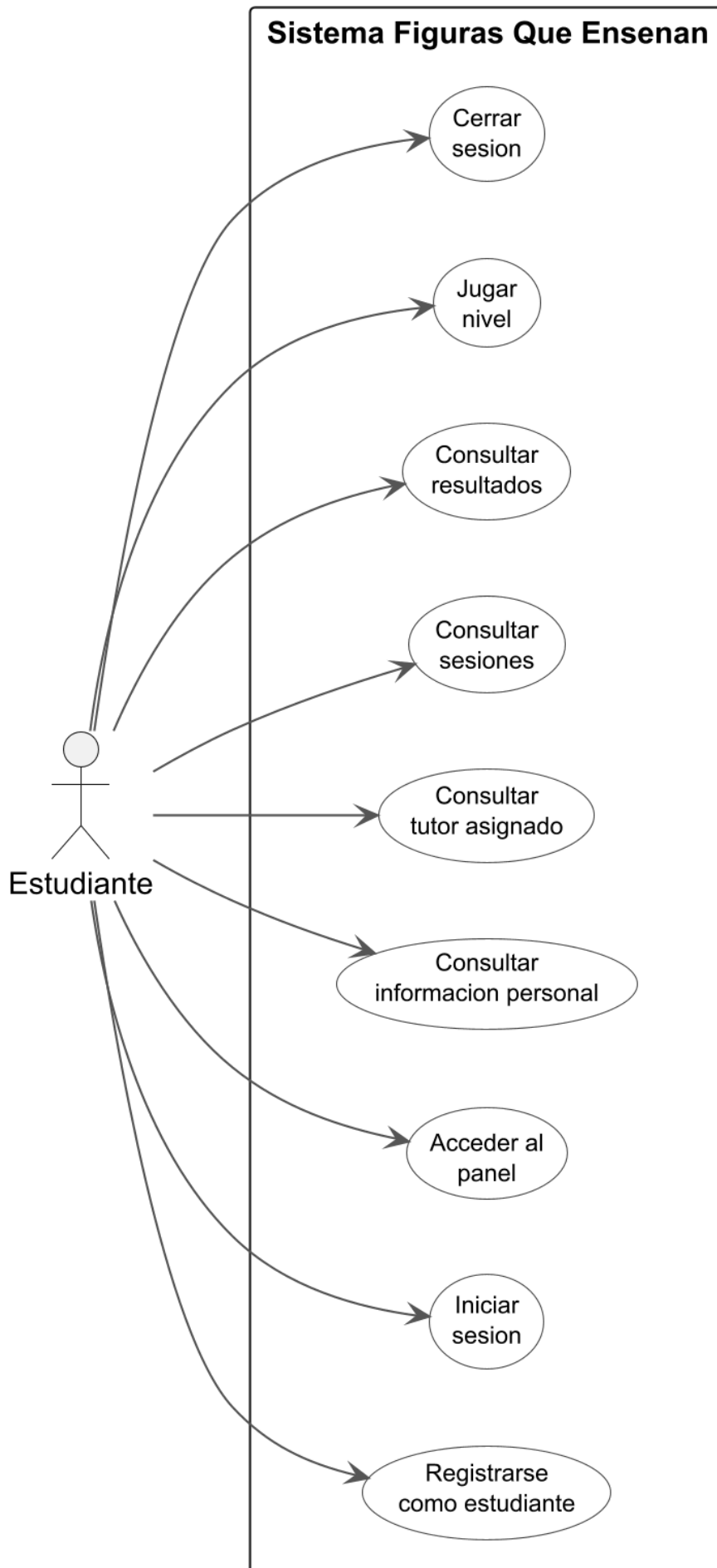
Actor	Descripcion
Estudiante	Usuario principal de la experiencia educativa. Puede consultar su informacion, su tutor asignado, sus sesiones y sus resultados de actividad.
Profesor / Tutor / Padre	Usuario responsable del seguimiento de estudiantes. Puede consultar su perfil, asociar estudiantes y revisar informacion, sesiones y resultados de los alumnos asignados.
Administrador	Usuario encargado de supervisar la operacion general del sistema. Puede iniciar sesion en el cliente administrativo, consultar estadisticas, revisar logs, monitorear servicios y gestionar su informacion administrativa.

3.2. Diagramas de Casos de Uso

Para mejorar la lectura, los casos de uso se presentan en diagramas separados por actor.

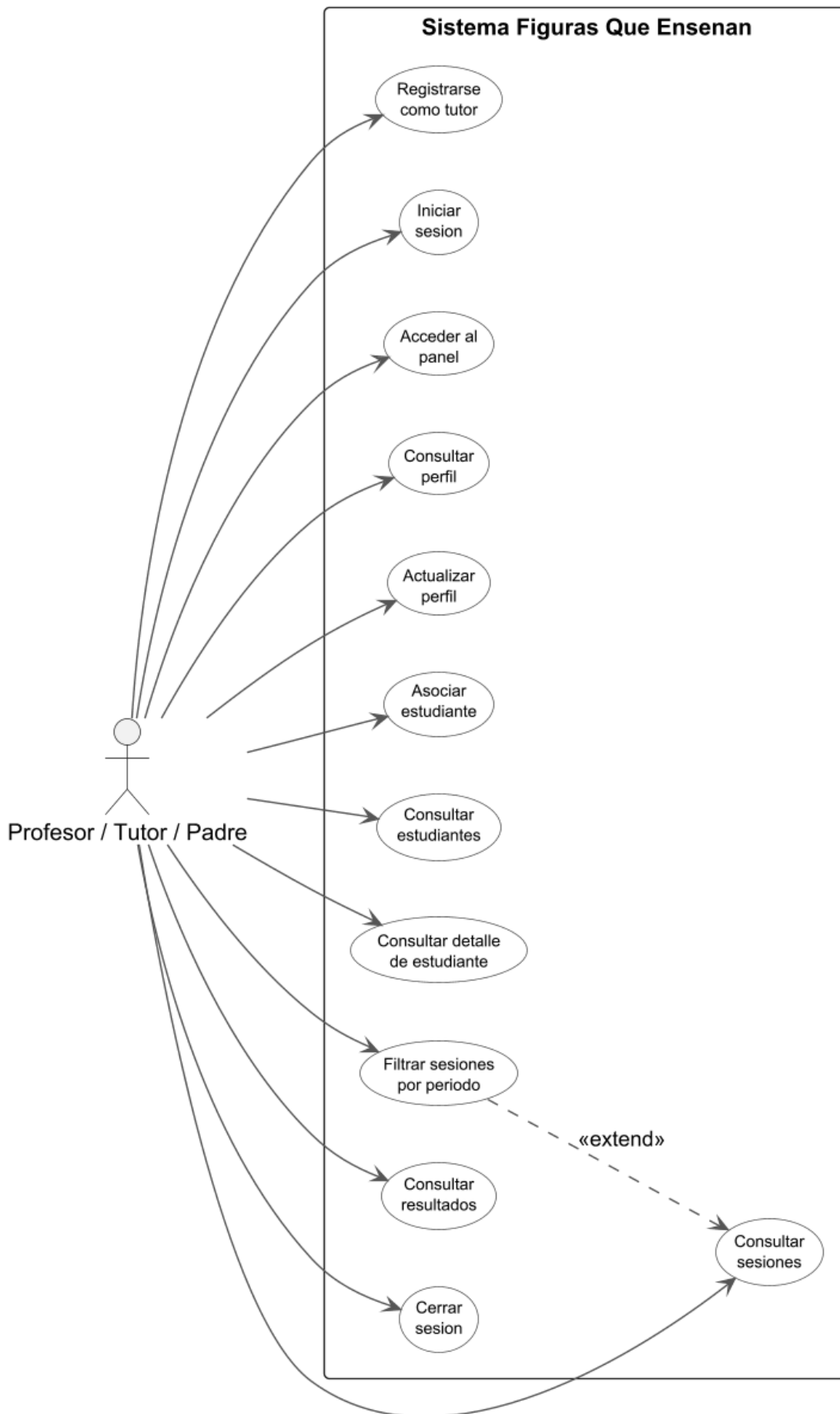
3.2.1. Estudiante

Casos de Uso - Estudiante



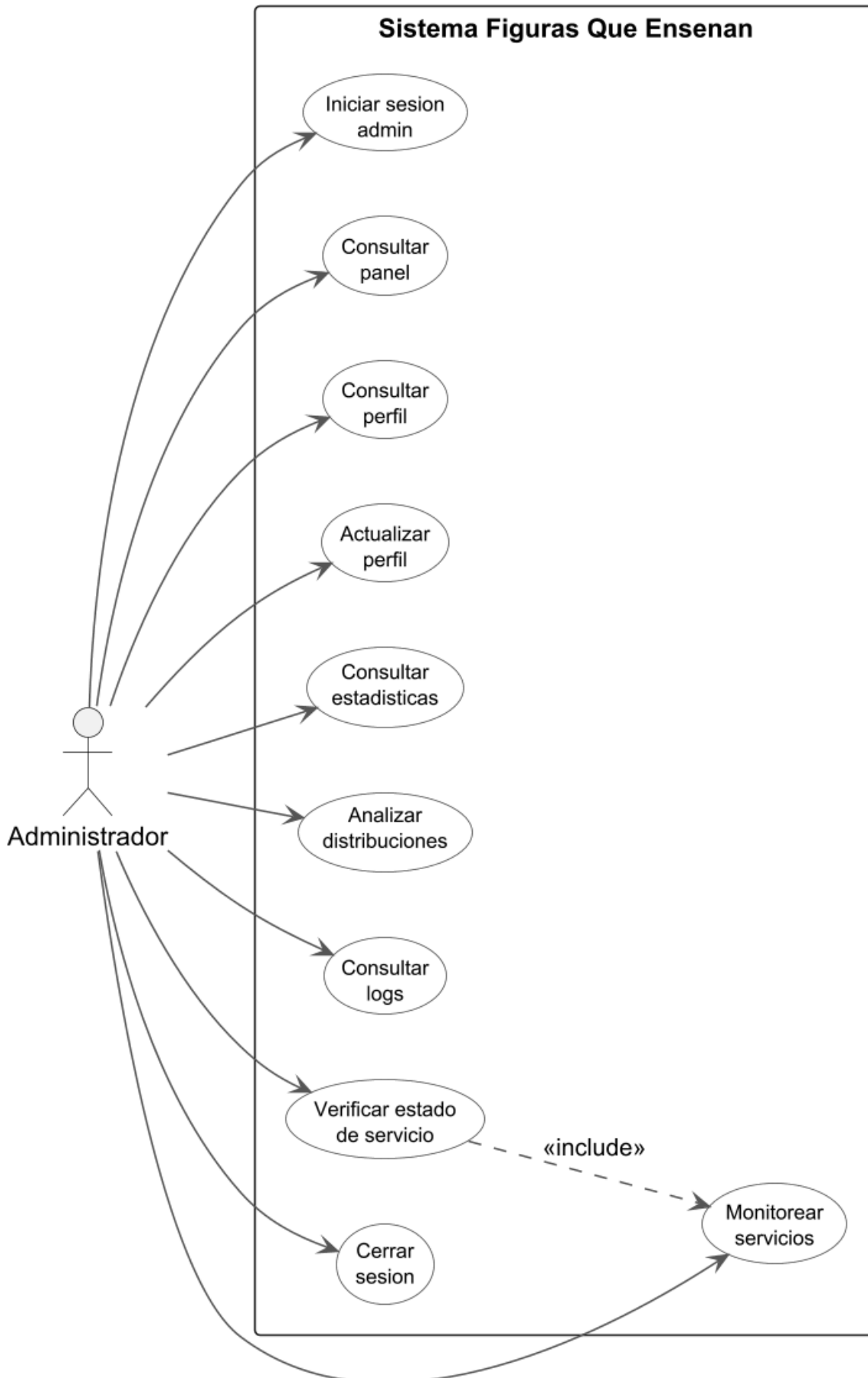
3.2.2. Profesor, Tutor o Padre

Casos de Uso - Profesor / Tutor / Padre



3.2.3. Administrador

Casos de Uso - Administrador



3.3. Casos de Uso del Estudiante

ID	Caso de uso	Descripcion
CU-E-01	Registrarse como estudiante	El estudiante puede crear una cuenta proporcionando sus datos personales y credenciales de acceso. El registro permite identificarlo dentro del sistema y asociar posteriormente su actividad educativa.
CU-E-02	Iniciar sesion	El estudiante puede iniciar sesion con correo y contrasena. Al autenticarse, el sistema genera un token que permite reconocer su identidad y rol.
CU-E-03	Acceder al panel del estudiante	El sistema dirige al estudiante a su pantalla principal, donde puede consultar un resumen de su informacion, sesiones registradas y estado general de su actividad.
CU-E-04	Consultar informacion personal	El estudiante puede visualizar sus datos registrados, como nombre, edad, genero, pais, correo y condicion de neurodivergencia cuando aplique.
CU-E-05	Consultar tutor asignado	El estudiante puede revisar la informacion del tutor asociado a su cuenta, cuando exista una asignacion registrada.
CU-E-06	Consultar sesiones de uso	El estudiante puede consultar el historial de sesiones asociadas a su cuenta, incluyendo fechas, dispositivo y datos relacionados con la actividad.
CU-E-07	Consultar resultados de niveles	El estudiante puede consultar resultados asociados a sus sesiones, como niveles completados, intentos, fallos, tiempo de finalizacion y estado de completado.
CU-E-08	Jugar nivel	El estudiante puede realizar una actividad o nivel dentro de la aplicacion. Durante este proceso, el sistema registra la sesion de uso y guarda los resultados obtenidos, como intentos, fallos, tiempo de finalizacion y estado de completado.
CU-E-09	Cerrar sesion	El estudiante puede cerrar su sesion para finalizar el acceso activo a la aplicacion.

3.4. Casos de Uso del Profesor, Tutor o Padre

ID	Caso de uso	Descripcion
CU-T-01	Registrarse como tutor	El profesor, tutor o padre puede crear una cuenta con sus datos personales, credenciales y grado o perfil correspondiente.
CU-T-02	Iniciar sesion	El tutor puede iniciar sesion con correo y contrasena. El sistema valida sus credenciales y lo dirige a las funciones correspondientes a su rol.

ID	Caso de uso	Descripcion
CU-T-03	Acceder al panel del tutor	El tutor puede consultar una vista principal con su informacion general y el numero de estudiantes asociados a su cuenta.
CU-T-04	Consultar perfil del tutor	El tutor puede visualizar sus datos personales y profesionales, como nombre, correo, pais, genero, edad, grado y fecha de registro.
CU-T-05	Actualizar perfil del tutor	El tutor puede modificar datos editables de su perfil, como pais, genero, edad o grado.
CU-T-06	Asociar estudiante	El tutor puede vincular un estudiante a su cuenta mediante el correo del alumno, siempre que la relacion sea valida.
CU-T-07	Consultar estudiantes asignados	El tutor puede ver la lista de estudiantes asociados a su cuenta y acceder al detalle de cada uno.
CU-T-08	Consultar detalle de estudiante	El tutor puede revisar la informacion individual de un estudiante asignado, incluyendo datos personales, tutor asignado y características registradas.
CU-T-09	Consultar sesiones de estudiantes	El tutor puede consultar las sesiones de uso de sus estudiantes asignados, incluyendo historial y datos de actividad.
CU-T-10	Filtrar sesiones por periodo	El tutor puede filtrar sesiones por rangos de tiempo, como hoy, ultimos dias, ultimo mes o todas las sesiones disponibles.
CU-T-11	Consultar resultados de estudiantes	El tutor puede revisar resultados de niveles asociados a los estudiantes bajo su seguimiento, incluyendo intentos, fallos, tiempo y niveles completados.
CU-T-12	Cerrar sesion	El tutor puede cerrar su sesion para terminar el acceso activo a la aplicacion.

3.5. Casos de Uso del Administrador

ID	Caso de uso	Descripcion
CU-A-01	Iniciar sesion como administrador	El administrador puede acceder al cliente administrativo mediante credenciales propias. El sistema valida que el usuario tenga rol de administrador.
CU-A-02	Consultar panel administrativo	El administrador puede acceder a una pantalla principal desde la que navega hacia cuenta, logs, monitoreo de servicios y estadísticas.
CU-A-03	Consultar perfil administrativo	El administrador puede visualizar los datos de su cuenta, como nombre, correo, telefono, usuario y fecha de registro.

ID	Caso de uso	Descripcion
CU-A-04	Actualizar perfil administrativo	El administrador puede modificar informacion de su cuenta, incluyendo datos personales y contrasena cuando sea necesario.
CU-A-05	Consultar estadisticas generales	El administrador puede revisar un resumen estadistico con totales de estudiantes, tutores, administradores, usuarios registrados, actividad reciente, horas jugadas y niveles superados.
CU-A-06	Analizar distribuciones de usuarios	El administrador puede consultar metricas agrupadas por genero de estudiantes, neurodivergencia de estudiantes, grado de tutores y genero de tutores.
CU-A-07	Consultar logs del sistema	El administrador puede revisar eventos generados por los servicios para analizar actividad, errores, rutas consultadas y acciones realizadas. Esta consulta puede realizarse mediante filtros y tambien puede mostrar eventos en tiempo real.
CU-A-08	Monitorear servicios	El administrador puede consultar tarjetas de servicios conocidos para revisar su estado general y rutas de verificacion.
CU-A-09	Verificar estado de un servicio	El administrador puede ejecutar una consulta de salud sobre servicios configurados para validar si responden correctamente.
CU-A-10	Cerrar sesion	El administrador puede cerrar su sesion en el cliente administrativo.

3.6. Descripcion Detallada de Casos de Uso Principales

Los siguientes casos de uso detallan los flujos mas importantes del sistema y se derivan de los casos ya definidos en las tablas anteriores. Cuando una misma funcion existe para mas de un rol, se agrupa en una sola descripcion para evitar repetir el mismo comportamiento tecnico.

3.6.1. CU Principal 01: Iniciar sesion

Elemento	Descripcion
Actores	Estudiante, Profesor / Tutor / Padre y Administrador.
Precondiciones	El usuario debe existir en el sistema. El API Gateway, el microservicio de autenticacion y PostgreSQL deben estar disponibles. El cliente debe conocer la URL del Gateway.
Postcondiciones	El usuario queda autenticado. El cliente conserva el token JWT recibido y puede consumir las funciones permitidas por su rol.

Elemento	Descripcion
Flujo principal	1. El actor ingresa correo y contrasena en el cliente correspondiente. 2. El cliente envia la solicitud al API Gateway. 3. El Gateway redirige la peticion al microservicio de autenticacion. 4. El microservicio valida las credenciales contra PostgreSQL. 5. Si las credenciales son correctas, el sistema genera un token JWT con identidad y rol. 6. El token se devuelve al cliente. 7. El cliente dirige al usuario hacia las funciones permitidas para su rol.
Flujos alternos	• Si el actor es estudiante o tutor, el inicio de sesion se realiza mediante el flujo general de autenticacion. • Si el actor es administrador, el inicio de sesion se realiza mediante el flujo administrativo y el sistema valida que el usuario tenga rol de administrador.
Flujos de excepcion	• Si las credenciales son incorrectas, el sistema rechaza el acceso. • Si el usuario no existe, el sistema responde con error de autenticacion. • Si el token no puede generarse, el sistema no inicia sesion. • Si el Gateway, el servicio de autenticacion o PostgreSQL no estan disponibles, el cliente muestra un error de conexion o servicio.

3.6.2. CU Principal 02: Registrar usuario

Elemento	Descripcion
Actores	Estudiante y Profesor / Tutor / Padre.
Precondiciones	El usuario no debe estar registrado previamente con el mismo correo. El API Gateway, el microservicio de autenticacion y PostgreSQL deben estar disponibles.
Postcondiciones	Se crea la cuenta del usuario con el rol correspondiente. El usuario queda disponible para iniciar sesion o continuar el flujo de autenticacion definido por el cliente.

Elemento	Descripcion
Flujo principal	1. El actor abre el formulario de registro. 2. El actor captura sus datos personales y credenciales. 3. El cliente envia la informacion al API Gateway. 4. El Gateway redirige la solicitud al microservicio de autenticacion. 5. El microservicio valida los datos recibidos. 6. El sistema verifica que el correo no exista previamente. 7. El sistema guarda la cuenta en PostgreSQL con el rol correspondiente. 8. El sistema responde confirmando el registro.
Flujos alternos	• Si el actor se registra como estudiante, la cuenta se crea con el rol de estudiante. • Si el actor se registra como tutor, profesor o padre, la cuenta se crea con el rol de tutor.
Flujos de excepcion	• Si faltan campos obligatorios, el sistema rechaza la solicitud. • Si el formato de los datos no es valido, el sistema devuelve error de validacion. • Si el correo ya existe, el sistema no crea una cuenta duplicada. • Si PostgreSQL no esta disponible, el sistema no registra la cuenta.

3.6.3. CU Principal 03: Consultar progreso del estudiante

Elemento	Descripcion
Actores	Estudiante y Profesor / Tutor / Padre.
Precondiciones	El actor debe haber iniciado sesion. El token JWT debe ser valido. El Gateway, el microservicio de datos y PostgreSQL deben estar disponibles.
Postcondiciones	El actor visualiza informacion de progreso permitida por su rol. No se modifican datos al realizar la consulta.
Flujo principal	1. El actor solicita consultar informacion de progreso. 2. El cliente envia la peticion al API Gateway con el token JWT. 3. El Gateway redirige la solicitud al microservicio de datos. 4. El microservicio valida la identidad y el rol del actor. 5. El servicio consulta PostgreSQL. 6. El sistema devuelve informacion de perfil, tutor, estudiantes asignados, sesiones o resultados segun corresponda. 7. El cliente muestra la informacion al actor.

Elemento	Descripcion
Flujos alternos	• Si el actor es estudiante, solo puede consultar informacion asociada a su propia cuenta. • Si el actor es tutor, profesor o padre, solo puede consultar estudiantes asociados a su cuenta. • Si el tutor filtra sesiones por periodo, el sistema limita los resultados al rango solicitado.
Flujos de excepcion	• Si el token esta ausente, expirado o es invalido, el sistema rechaza la consulta. • Si el actor intenta consultar informacion fuera de sus permisos, el sistema niega el acceso. • Si no existen registros de sesiones o resultados, el sistema devuelve una respuesta vacia o sin datos. • Si el microservicio de datos o PostgreSQL no estan disponibles, el cliente muestra un error de servicio.

3.6.4. CU Principal 04: Jugar nivel

Elemento	Descripcion
Actores	Estudiante.
Precondiciones	El estudiante debe haber iniciado sesion. El cliente debe contar con un token JWT valido. El Gateway, el microservicio de datos y PostgreSQL deben estar disponibles. Para actividades con deteccion de manos, tambien debe estar disponible HandsDetection.
Postcondiciones	La actividad queda reflejada como sesion de uso y resultado de nivel cuando el flujo se completa correctamente. El avance puede consultarse posteriormente por el estudiante o por su tutor.

Elemento	Descripcion
Flujo principal	1. El estudiante inicia un nivel o actividad desde el cliente. 2. El cliente ejecuta la experiencia interactiva del nivel. 3. Si la actividad requiere deteccion de dedos, el cliente envia la imagen al API Gateway mediante la ruta <code>/hands</code>. 4. El Gateway envia la imagen al servicio HandsDetection mediante gRPC. 5. HandsDetection procesa la imagen y devuelve el conteo de dedos detectado. 6. El cliente utiliza la respuesta para continuar la actividad. 7. Al finalizar, el cliente envia los datos de sesion o resultado al API Gateway. 8. El Gateway redirige la informacion al microservicio de datos. 9. El microservicio valida la solicitud y guarda la sesion o resultado en PostgreSQL. 10. El sistema confirma el registro del avance.
Flujos alternos	• Si el nivel no requiere deteccion de manos, el flujo continua sin invocar HandsDetection. • Si el estudiante no completa el nivel, el sistema puede conservar solo la informacion de sesion o no registrar resultado completado, segun la informacion enviada por el cliente.
Flujos de excepcion	• Si el token del estudiante es invalido, el sistema rechaza el registro. • Si HandsDetection no responde, la actividad que depende de deteccion de manos no puede obtener el conteo esperado. • Si los datos de sesion o resultado son invalidos, el microservicio de datos rechaza la solicitud. • Si PostgreSQL no esta disponible, el avance no puede persistirse.

3.6.5. CU Principal 05: Consultar logs y monitorear servicios

Elemento	Descripcion
Actores	Administrador.
Precondiciones	El administrador debe haber iniciado sesion en el cliente WPF. El token JWT debe ser valido. El Gateway y los servicios consultados deben estar disponibles para responder.
Postcondiciones	El administrador visualiza logs, eventos o estado de servicios. La consulta no modifica informacion del sistema.

Elemento	Descripcion
Flujo principal	1. El administrador abre el cliente WPF. 2. El administrador inicia sesion mediante el API Gateway. 3. El administrador entra a la seccion de logs o monitoreo. 4. El WPF consulta al API Gateway usando el token JWT. 5. Para logs, el Gateway redirige la consulta al Logs Service. 6. El Logs Service consulta los eventos almacenados en mongo-logs. 7. Para monitoreo, el Gateway consulta el endpoint de salud del servicio solicitado. 8. El WPF muestra logs, filtros, eventos en tiempo real o tarjetas de estado segun la accion seleccionada.
Flujos alternos	• El administrador puede consultar logs generales o filtrarlos por servicio. • El administrador puede recibir eventos de logs en tiempo real mediante WebSocket. • El administrador puede consultar el estado de Gateway, Auth Service, microservicio de datos, Logs Service, RabbitMQ Mongo Listener, MongoDB, PostgreSQL y HandsDetection.
Flujos de excepcion	• Si el token del administrador es invalido, el sistema rechaza la consulta. • Si Logs Service no esta disponible, el WPF no puede cargar los logs. • Si mongo-logs no esta disponible, Logs Service no puede recuperar eventos. • Si RabbitMQ o el listener estan detenidos, pueden dejar de persistirse nuevos eventos. • Si un servicio no responde al monitoreo, el WPF lo muestra como no disponible o con error.

Chapter 4. Microservicios del Sistema

El proyecto se organiza en varios microservicios que separan responsabilidades del backend. Esta separacion permite que la autenticacion, la gestion de datos, la consulta de logs, la integracion por mensajeria y el acceso unificado desde clientes funcionen como piezas independientes, cada una con una funcion especifica dentro del sistema.

En esta seccion no se consideran los clientes de usuario, como la aplicacion web, la aplicacion WPF o la aplicacion Android. El enfoque se centra solamente en los servicios backend que exponen APIs, documentacion Swagger o endpoints tecnicos de consulta.

4.1. API Gateway

El API Gateway funciona como punto de entrada unificado para las operaciones principales del sistema. Su responsabilidad es recibir solicitudes de los clientes, redirigirlas hacia los servicios correspondientes y presentar una interfaz centralizada para autenticacion, datos, deteccion de manos, estado de servicios y consulta de logs.

Tecnologia utilizada: Node.js con Express.

Este servicio es consumido principalmente por los clientes del sistema, ya que evita que cada cliente tenga que conocer directamente la ubicacion interna de cada microservicio. Tambien puede ser consultado por el administrador cuando se requiere revisar informacion operativa, salud de servicios o logs centralizados.

Metodo	Ruta	Funcion
POST	/auth/login	Iniciar sesion como estudiante o tutor.
POST	/auth/admin/login	Iniciar sesion como administrador.
POST	/auth/register	Registrar un usuario y devolver un token JWT.
GET	/data/admins	Consultar administradores registrados.
POST	/data/admins	Crear un administrador.
GET	/data/admins/{id}	Consultar un administrador por identificador.
PUT	/data/admins/{id}	Actualizar informacion de un administrador.
DELETE	/data/admins/{id}	Eliminar un administrador.
GET	/data/students	Consultar estudiantes.
GET	/data/students/{id}	Consultar un estudiante por identificador.
PUT	/data/students/{id}	Actualizar informacion de un estudiante.
GET	/data/students/{id} /sessions	Consultar sesiones asociadas a un estudiante.
POST	/data/sessions	Registrar una sesion de actividad.
POST	/data/level-results	Registrar un resultado de nivel.
GET	/data/tutors/{id}	Consultar un tutor por identificador.

Metodo	Ruta	Funcion
PUT	/data/tutors/{id}	Actualizar informacion de un tutor.
GET	/data/tutors/{id}/students	Consultar estudiantes asignados a un tutor.
POST	/data/tutors/assign-student	Asignar un estudiante a un tutor.
GET	/data/dashboard/summary	Consultar resumen estadistico del sistema.
POST	/hands	Enviar una imagen para deteccion de manos.
GET	/logs/api/logs	Consultar logs agregados.
GET	/logs/api/logs/service/{serviceName}	Consultar logs filtrados por servicio.
GET	/health	Consultar el estado del gateway.
GET	/health/{serviceName}	Consultar el estado de un servicio especifico.

El API Gateway cuenta con documentacion interactiva Swagger/OpenAPI. Esta documentacion permite visualizar las rutas disponibles, los metodos HTTP, los parametros esperados, los esquemas de respuesta y los endpoints que requieren autorizacion.

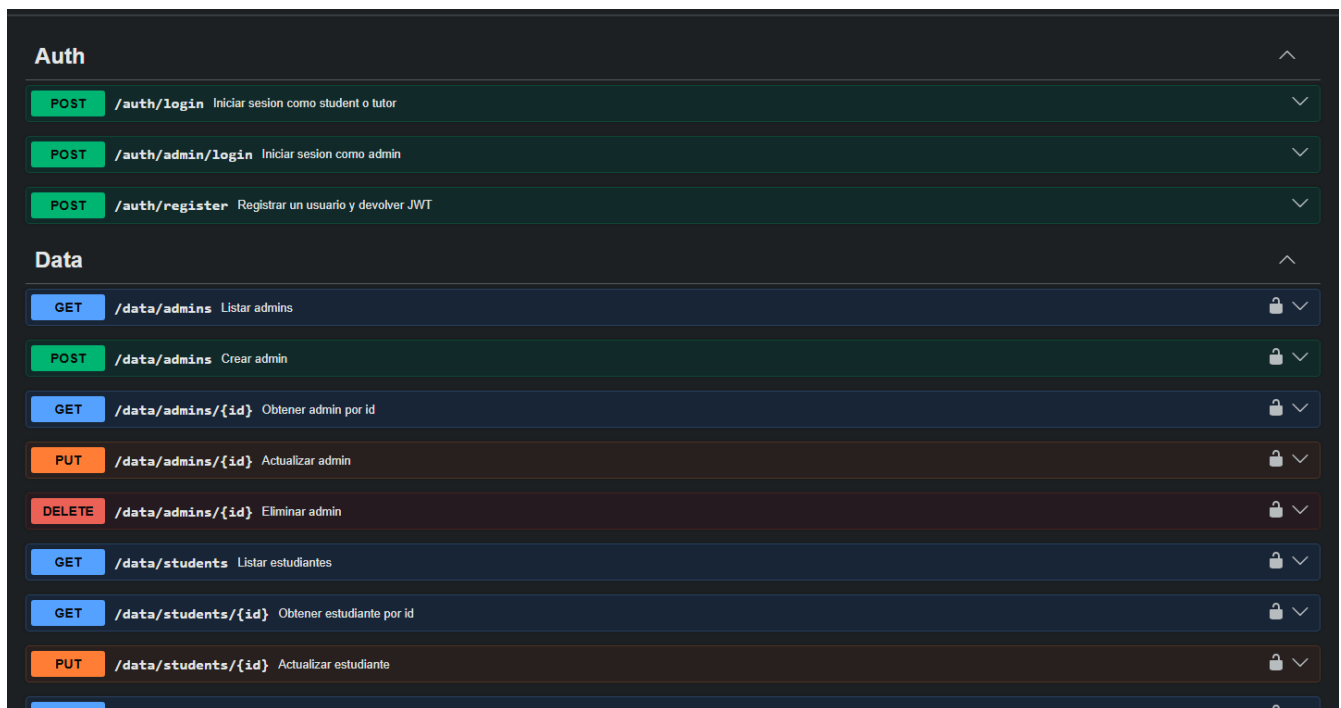


Figure 1. Evidencia Swagger del API Gateway

4.2. Microservicio de Autenticacion

El microservicio de autenticacion concentra las operaciones relacionadas con inicio de sesion, registro de usuarios y emision de tokens JWT. Su funcion principal es validar credenciales y entregar al resto del sistema una identidad autenticada con el rol correspondiente.

Tecnologia utilizada: .netcore con ASP.NET Core.

Este servicio es consumido por el API Gateway y por cualquier cliente que necesite iniciar sesion o registrar usuarios. Los tokens generados por este microservicio son utilizados posteriormente para controlar el acceso a las funcionalidades protegidas del sistema.

Metodo	Ruta	Funcion
POST	/auth/login	Validar credenciales de estudiante o tutor.
POST	/auth/admin/login	Validar credenciales de administrador.
POST	/auth/register	Registrar un nuevo usuario y generar su respuesta de autenticacion.
GET	/health	Consultar el estado del servicio y su conexion con la base de datos.

El microservicio expone documentacion Swagger para consultar sus contratos de entrada y salida. En esta interfaz se muestran las operaciones de autenticacion y los modelos usados para login y registro.

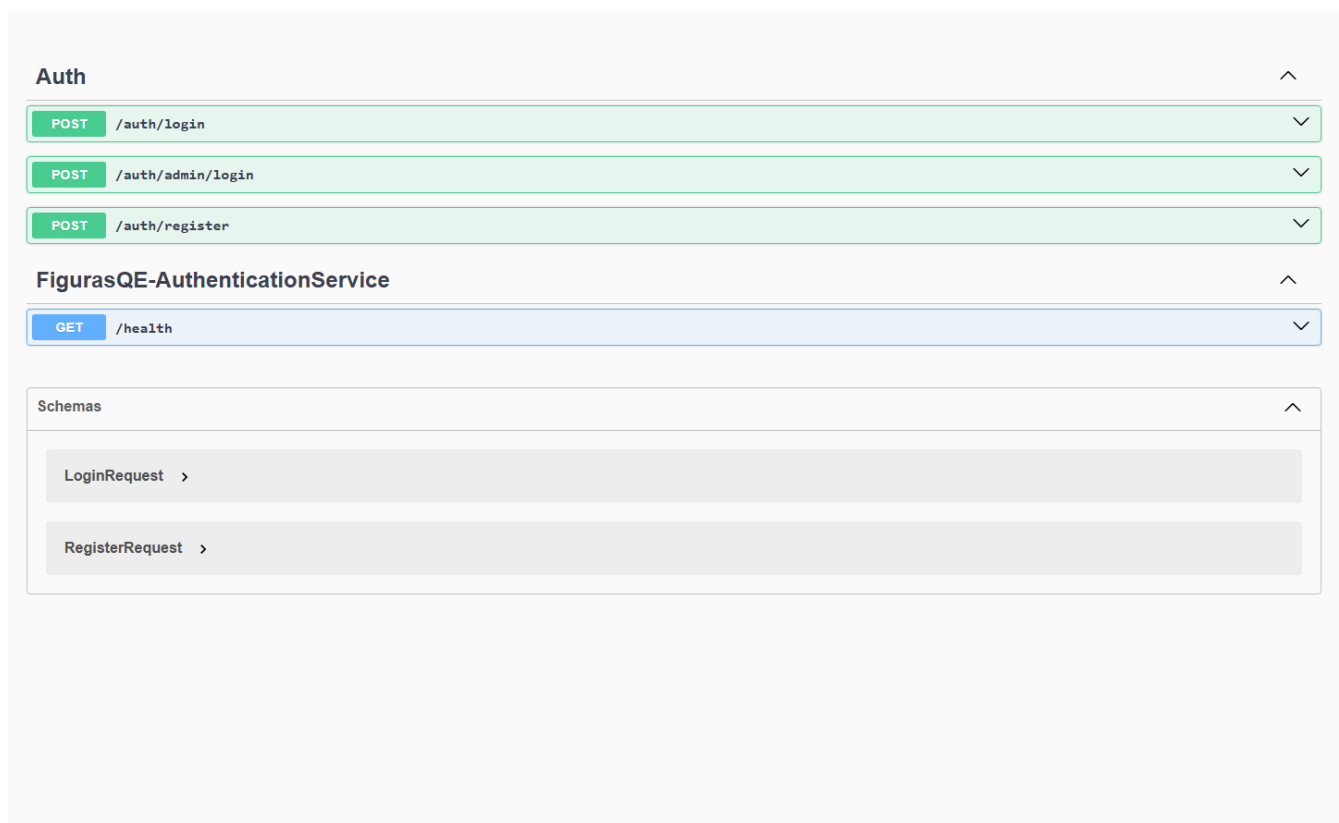


Figure 2. Evidencia Swagger del Microservicio de Autenticacion

4.3. Microservicio de Datos

El microservicio de datos administra la informacion relacional del dominio principal del sistema. Su funcion es exponer operaciones para administradores, estudiantes, tutores, niveles, sesiones, resultados de nivel y resúmenes estadísticos.

Tecnologia utilizada: .netcore con ASP.NET Core y Entity Framework Core.

Este servicio es consumido por el API Gateway y consultado indirectamente por estudiantes, tutores, profesores y administradores. El estudiante interactúa con información de su progreso y sesiones; el tutor o profesor consulta estudiantes asignados y resultados; el administrador consulta información general y estadísticas.

Metodo	Ruta	Funcion
GET	/admins	Consultar administradores.
POST	/admins	Crear administradores.
GET	/admins/{id}	Consultar un administrador por identificador.
PUT	/admins/{id}	Actualizar un administrador.
DELETE	/admins/{id}	Eliminar un administrador.
GET	/students	Consultar estudiantes.
POST	/students	Crear estudiantes.
GET	/students/{id}	Consultar un estudiante por identificador.
PUT	/students/{id}	Actualizar un estudiante.
DELETE	/students/{id}	Eliminar un estudiante.
GET	/students/{id}/tutor	Consultar el tutor asociado a un estudiante.
GET	/students/tutor/{tutorId}/ids	Consultar identificadores de estudiantes asociados a un tutor.
GET	/students/{studentId}/sessions	Consultar sesiones de un estudiante.
GET	/students/{studentId}/sessions/ids	Consultar identificadores de sesiones de un estudiante.
GET	/tutors	Consultar tutores.
POST	/tutors	Crear tutores.
GET	/tutors/{id}	Consultar un tutor por identificador.
PUT	/tutors/{id}	Actualizar un tutor.
DELETE	/tutors/{id}	Eliminar un tutor.
GET	/tutors/{id}/students	Consultar estudiantes asignados a un tutor.
GET	/tutors/{tutorId}/sessions	Consultar sesiones asociadas a estudiantes de un tutor.
POST	/tutors/assign-student	Asignar un estudiante a un tutor.
GET	/levels	Consultar niveles.
POST	/levels	Crear niveles.
GET	/levels/{id}	Consultar un nivel por identificador.

Metodo	Ruta	Funcion
PUT	/levels/{id}	Actualizar un nivel.
DELETE	/levels/{id}	Eliminar un nivel.
GET	/sessions	Consultar sesiones.
POST	/sessions	Crear sesiones.
GET	/sessions/{id}	Consultar una sesion por identificador.
PUT	/sessions/{id}	Actualizar una sesion.
DELETE	/sessions/{id}	Eliminar una sesion.
GET	/sessions/{sessionId}/level-results/ids	Consultar identificadores de resultados asociados a una sesion.
GET	/level-results	Consultar resultados de nivel.
POST	/level-results	Crear resultados de nivel.
GET	/level-results/{id}	Consultar un resultado de nivel.
PUT	/level-results/{id}	Actualizar un resultado de nivel.
DELETE	/level-results/{id}	Eliminar un resultado de nivel.
GET	/dashboard/summary	Consultar resumen estadistico.
GET	/health	Consultar el estado del servicio y su conexion con la base de datos.

El microservicio de datos cuenta con documentacion Swagger para explorar las rutas del dominio educativo y probar las operaciones disponibles. La documentacion muestra tambien los endpoints protegidos que requieren autentificacion mediante token.

MicroservicioFiguras	
GET	/health
POST	/admins
GET	/admins
GET	/students
POST	/students
GET	/students/{id}
PUT	/students/{id}
DELETE	/students/{id}
GET	/students/{id}/tutor
GET	/students/tutor/{tutorId}/ids
GET	/students/{studentId}/sessions/ids
GET	/tutors
POST	/tutors
GET	/tutors/{id}
PUT	/tutors/{id}

Figure 3. Evidencia Swagger del Microservicio de Datos

4.4. Servicio de Deteccion de Manos

El servicio HandsDetection procesa imagenes para detectar manos y calcular el conteo de dedos por mano. Su funcion dentro del sistema es atender la capacidad de reconocimiento visual usada por los clientes a traves del API Gateway.

Tecnologia utilizada: Python con gRPC, MediaPipe y OpenCV.

Este servicio no se consume directamente desde los clientes. El cliente envia la solicitud HTTP al Gateway mediante la ruta `/hands` y el Gateway traduce esa operacion hacia el servicio gRPC `FingerDetector.StreamFrames`. En Docker el contenedor se llama `fqe-hands-detection` y publica el puerto `50051`. Ademas, dentro del contenedor levanta un endpoint HTTP tecnico en `127.0.0.1:8080/health`, utilizado por Docker para validar su estado.

Protocolo	Operacion	Funcion
gRPC	FingerDetector.StreamFrames	Recibir imagenes codificadas y devolver conteo de dedos por mano.
HTTP	GET /health	Verificar internamente que el proceso del servicio esta activo.

El contrato de comunicacion se define en `fingers.proto`. La peticion usa `FrameRequest`, que contiene la imagen y el identificador de cliente; la respuesta usa `FingerResponse`, que incluye conteo de mano izquierda, mano derecha, total y bandera de manos detectadas. A diferencia de los servicios HTTP, este componente no expone Swagger porque su interfaz principal es gRPC.

4.5. Microservicio de Logs

El microservicio de logs permite consultar eventos operativos generados por los servicios del sistema. Su funcion es recuperar registros almacenados en MongoDB, aplicar filtros de busqueda y facilitar la supervision del comportamiento del backend.

Tecnologia utilizada: Node.js con Express, MongoDB Driver y WebSocket.

Este servicio es consultado principalmente por el administrador, ya sea directamente o a traves del API Gateway. Tambien sirve como soporte tecnico para analizar errores, rutas consultadas, acciones ejecutadas, servicios involucrados y tiempos de respuesta.

Metodo	Ruta	Funcion
GET	/api/logs	Consultar logs generales con filtros opcionales.
GET	/api/logs/service/{serviceName}	Consultar logs filtrados por servicio.
GET	/health	Consultar el estado del servicio y su conexion con MongoDB.
WS	/ws/logs	Recibir eventos de logs en tiempo real.

La documentacion Swagger del microservicio muestra las rutas HTTP disponibles para consulta de logs y salud del servicio. El canal WebSocket se documenta como parte de la descripcion funcional del servicio, ya que OpenAPI esta orientado principalmente a endpoints HTTP.

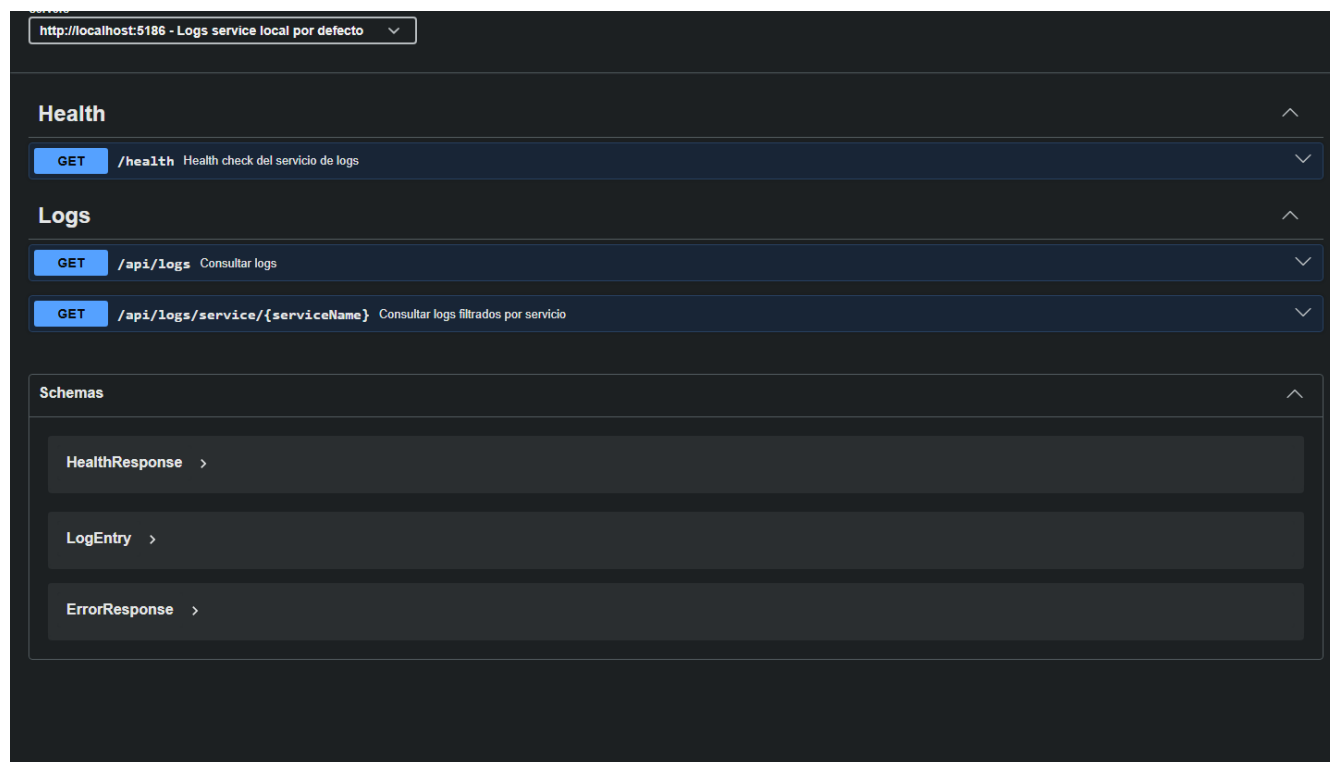


Figure 4. Evidencia Swagger del Microservicio de Logs

4.6. Listener RabbitMQ-MongoDB

El listener RabbitMQ-MongoDB se encarga de consumir eventos publicados en la cola de mensajería del sistema y persistirlos en MongoDB. Su responsabilidad es separar la generación de logs de su almacenamiento, permitiendo que los servicios productores no dependan directamente de la base documental.

Tecnología utilizada: Node.js con amqp-lib, MongoDB Driver y Express para su endpoint técnico de monitoreo.

Este servicio no es consumido por usuarios finales. Es consultado por administradores o responsables técnicos solamente para verificar su estado operativo. En la arquitectura funciona como componente de integración entre la mensajería RabbitMQ y la persistencia documental de eventos.

Metodo	Ruta	Funcion
GET	/health	Consultar el estado del listener, su conexión con RabbitMQ y su conexión con MongoDB.

El listener también expone una documentación Swagger sencilla enfocada en su endpoint de verificación de salud. Esto permite comprobar que el componente cuenta con una interfaz técnica documentada para monitoreo.

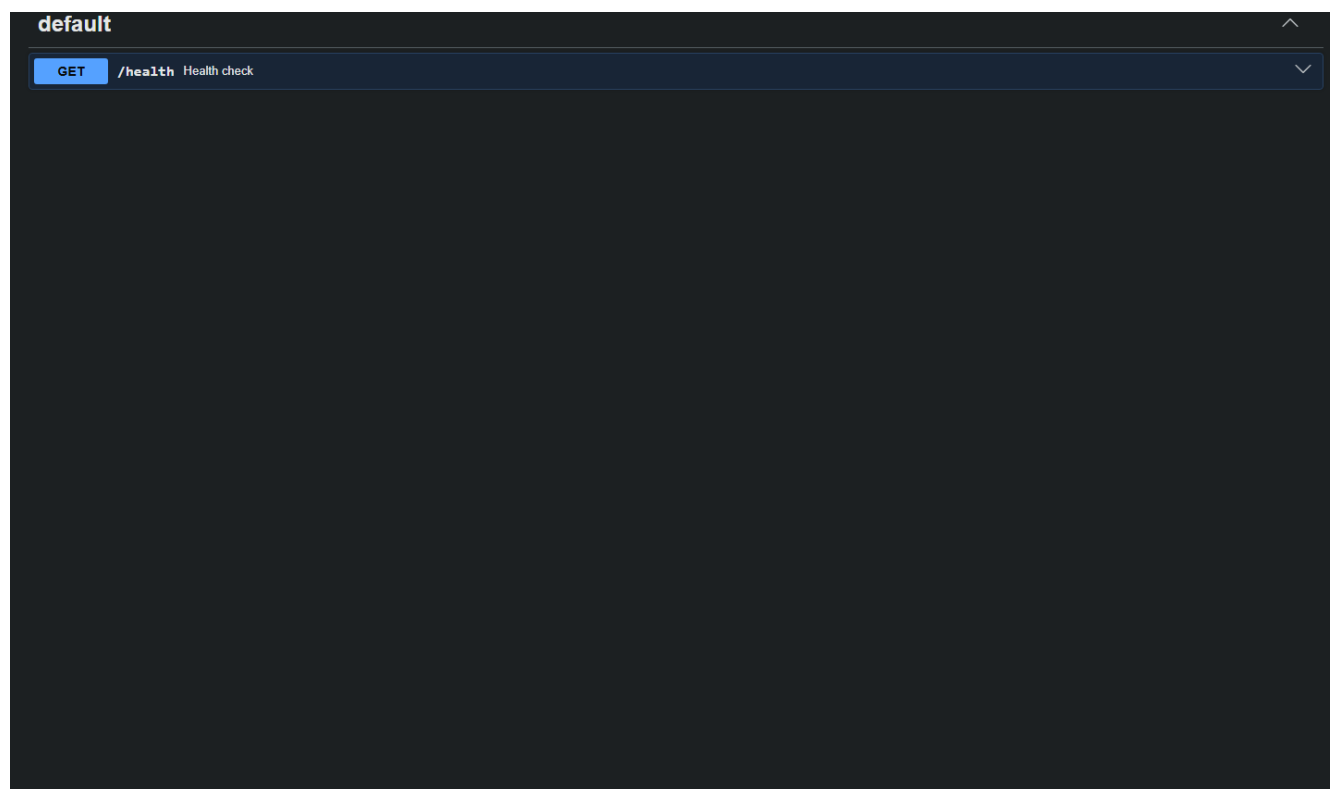


Figure 5. Evidencia Swagger del Listener RabbitMQ-MongoDB

Chapter 5. Clientes del Sistema

El sistema cuenta con diferentes clientes de usuario que consumen las funcionalidades expuestas por los microservicios. Cada cliente esta orientado a un contexto de uso distinto: acceso web para estudiantes, tutores o profesores; una aplicacion de escritorio para administracion; y una aplicacion movil para consulta y seguimiento desde dispositivos Android.

La separacion entre clientes permite adaptar la experiencia de usuario al tipo de actor que interactua con el sistema, sin mezclar responsabilidades de interfaz con la logica de los microservicios.

5.1. Cliente Web

El cliente web corresponde a la interfaz accesible desde navegador. Esta orientado principalmente a usuarios como estudiantes, tutores o profesores, permitiendo iniciar sesion, registrarse y consultar informacion asociada al perfil autenticado.

Tecnologia utilizada: .netcore con ASP.NET Core Razor Pages.

Este cliente consume principalmente el API Gateway, enviando solicitudes de autenticacion y consultas de datos mediante HTTP. Despues del inicio de sesion, conserva el token JWT dentro de la sesion del usuario y lo utiliza para consultar informacion protegida.

Aspecto	Descripcion
Usuarios principales	Estudiantes, tutores y profesores.
Funciones principales	Inicio de sesion, registro de usuario, consulta de perfil, consulta de informacion de estudiante y consulta de informacion de tutor.
Servicios consumidos	API Gateway para autenticacion y acceso a datos.
Manejo de sesion	Uso de autenticacion por cookies en el cliente y almacenamiento del token JWT como claim de sesion.

La eleccion de Razor Pages se justifica porque permite construir una interfaz web tradicional, organizada por paginas, con integracion directa entre vistas, modelos y logica de servidor. Para este proyecto resulta conveniente porque las pantallas de estudiante, tutor y autenticacion pueden manejarse de forma clara sin requerir una aplicacion web de una sola pagina.



Figure 6. Evidencia de interfaz del Cliente Web

5.2. Cliente WPF Administrativo

El cliente WPF corresponde a una aplicacion de escritorio dirigida al administrador del sistema. Su objetivo es proporcionar una interfaz enfocada en supervision, consulta de estadisticas, monitoreo de servicios, revision de logs y gestion del perfil administrativo.

Tecnologia utilizada: .netcore con WPF.

Este cliente consume el API Gateway y el modulo de logs expuesto por el backend. Utiliza el token de administrador para consultar informacion protegida, visualizar datos estadisticos y recibir eventos operativos del sistema.

Aspecto	Descripcion
Usuario principal	Administrador.
Funciones principales	Inicio de sesion administrativo, consulta de dashboard, consulta y actualizacion de perfil, consulta de logs, monitoreo de servicios y visualizacion de estadisticas.
Servicios consumidos	API Gateway y rutas de logs centralizadas.
Manejo de sesion	Estado de sesion administrativo en memoria, incluyendo token JWT y perfil del administrador.

La eleccion de WPF se justifica porque el rol administrativo requiere una interfaz mas cercana a

una herramienta de escritorio: paneles, tablas, filtros, monitoreo y vistas de control. WPF permite construir una aplicacion de escritorio para Windows con ventanas, vistas XAML y separacion entre controladores, servicios y modelos.

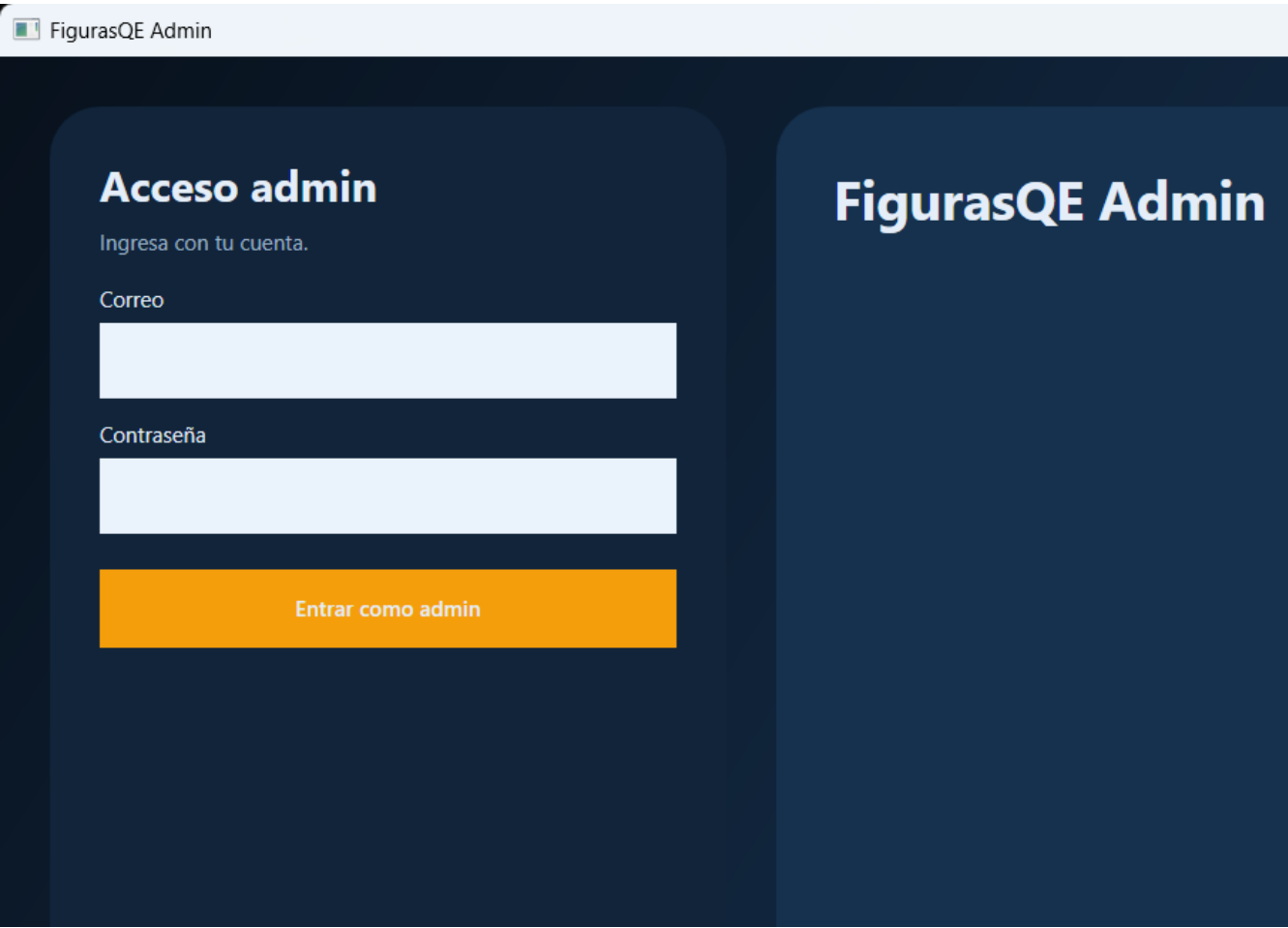


Figure 7. Evidencia de interfaz del Cliente WPF Administrativo

5.3. Cliente Android

El cliente Android corresponde a la aplicacion movil del sistema. Esta orientado a permitir acceso desde dispositivos Android, especialmente para tutores, profesores o usuarios que necesiten consultar informacion de estudiantes de forma portable.

Tecnologia utilizada: Kotlin con Android, Jetpack Compose, Retrofit, OkHttp y coroutines.

Este cliente consume el API Gateway mediante Retrofit. Implementa autenticacion, almacenamiento de sesion, navegacion entre pantallas, consulta de tutor, consulta de estudiantes asignados, detalle de estudiante, sesiones y actualizacion de perfil.

Aspecto	Descripcion
Usuarios principales	Tutores, profesores y estudiantes en escenarios moviles.
Funciones principales	Inicio de sesion, registro, navegacion por rol, consulta de perfil, consulta de estudiantes asignados, consulta de detalle de estudiante, consulta de sesiones y asignacion de estudiante a tutor.
Servicios consumidos	API Gateway para autenticacion y acceso a datos.

Aspecto	Descripcion
Manejo de sesion	Almacenamiento local del token JWT y lectura de claims para identificar al usuario autenticado.

La eleccion de Kotlin con Jetpack Compose se justifica porque permite desarrollar una interfaz nativa para Android con componentes modernos, declarativos y adaptados a dispositivos moviles. Retrofit y OkHttp facilitan el consumo de APIs HTTP documentadas por el backend, mientras que coroutines permiten manejar operaciones asincronas sin bloquear la interfaz.

La captura de evidencia del cliente Android queda pendiente de integrarse desde un emulador o dispositivo fisico, ya que la interfaz movil requiere ejecucion en un entorno Android para obtener una captura real de la aplicacion.

5.4. Justificacion General de los Clientes

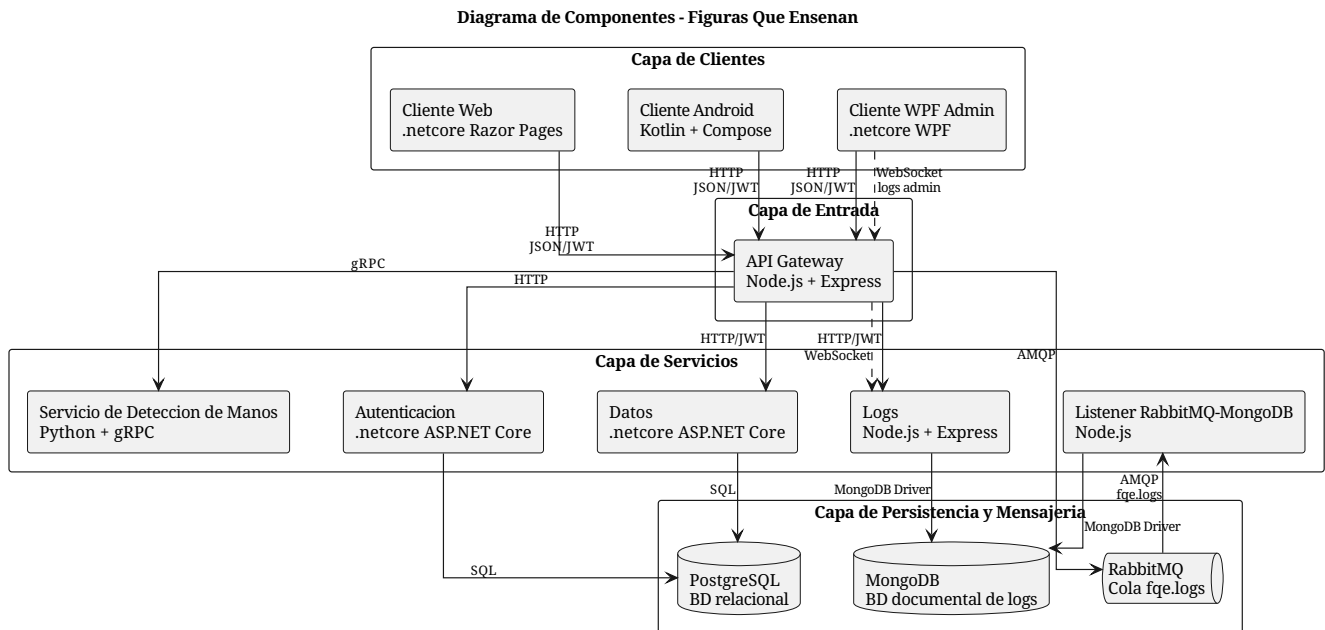
La existencia de varios clientes responde a que el sistema atiende necesidades distintas segun el actor:

- El cliente web facilita el acceso desde cualquier navegador y resulta adecuado para estudiantes, tutores o profesores que trabajan desde equipos de escritorio o portatiles.
- El cliente WPF concentra las funciones administrativas en una aplicacion de escritorio, adecuada para monitoreo, revision de informacion y operacion interna.
- El cliente Android permite consultar informacion desde dispositivos moviles, lo cual favorece el seguimiento por parte de tutores o profesores fuera de un entorno fijo.

En conjunto, los clientes consumen los servicios del backend sin duplicar la logica principal del sistema. La autenticacion, las reglas de acceso, la persistencia de datos y la consulta de logs permanecen en los microservicios, mientras que cada cliente se enfoca en presentar la informacion de forma adecuada para su usuario final.

Chapter 6. Diagrama de Componentes

El siguiente diagrama representa la arquitectura logica del sistema a nivel de componentes. Se muestra como los clientes consumen el sistema a traves del API Gateway, que servicios participan, que protocolos se utilizan para la comunicacion y que bases de datos son consumidas por cada servicio.



El API Gateway concentra el acceso desde los clientes y actua como intermediario hacia los microservicios. Los clientes no consumen directamente las bases de datos; toda consulta o modificacion pasa por servicios backend que aplican reglas de acceso, autenticacion y validacion.

PostgreSQL es consumido por los microservicios que manejan informacion relacional del dominio, como usuarios, administradores, tutores, estudiantes, sesiones, niveles y resultados. MongoDB es consumido por los componentes de logs, ya que almacena eventos operativos generados por los servicios. RabbitMQ permite desacoplar la generacion de eventos de su persistencia: el API Gateway publica eventos de log mediante AMQP y el listener RabbitMQ-MongoDB consume la cola **fqe.logs** para guardarlos en MongoDB.

6.1. Justificacion de la Arquitectura de Componentes

La arquitectura del sistema se organiza por capas para separar responsabilidades y evitar que los clientes dependan directamente de los detalles internos del backend. Esta organizacion permite identificar con claridad que componentes presentan la informacion, cuales reciben las solicitudes, cuales ejecutan la logica principal y cuales se encargan de persistir o transportar datos.

La capa de clientes agrupa las interfaces utilizadas por los usuarios finales. En esta capa se encuentran el cliente web, el cliente Android y el cliente WPF administrativo. Cada cliente esta construido con una tecnologia distinta porque responde a necesidades diferentes: acceso desde navegador, uso movil y administracion desde escritorio. Sin embargo, todos consumen el sistema a traves del mismo punto de entrada, lo que evita duplicar reglas de comunicacion con los microservicios.

La capa de entrada esta representada por el API Gateway. Este componente centraliza las solicitudes de los clientes y las redirige hacia el microservicio correspondiente. Su uso se justifica porque simplifica la comunicacion de los clientes, concentra rutas externas, permite aplicar mecanismos comunes de autenticacion y facilita la integracion con servicios internos que usan diferentes protocolos, como HTTP, WebSocket, AMQP y gRPC.

La capa de servicios contiene los componentes responsables de la logica funcional del sistema. El microservicio de autenticacion se encarga del inicio de sesion, registro y emision de tokens; el microservicio de datos administra la informacion educativa; el servicio de deteccion de manos procesa imagenes mediante una comunicacion gRPC; el servicio de logs permite consultar eventos del sistema; y el listener RabbitMQ-MongoDB procesa eventos asincronos para guardarlos en la base documental.

Separar los servicios permite que cada componente tenga una responsabilidad concreta. Esto reduce el acoplamiento entre partes del sistema y facilita el mantenimiento, ya que los cambios en autenticacion, datos, logs o deteccion de manos pueden manejarse de forma independiente. Tambien permite que cada servicio utilice la tecnologia o protocolo mas adecuado para su funcion.

La capa de persistencia y mensajeria contiene PostgreSQL, MongoDB y RabbitMQ. PostgreSQL se utiliza para los datos centrales del dominio porque estos tienen relaciones claras entre usuarios, tutores, estudiantes, sesiones, niveles y resultados. MongoDB se utiliza para los logs porque los eventos pueden variar en estructura y requieren almacenamiento flexible. RabbitMQ se incluye para desacoplar la publicacion de eventos de su almacenamiento, permitiendo que el sistema registre actividad sin bloquear el flujo principal de las solicitudes.

En conjunto, esta arquitectura favorece modularidad, separacion de responsabilidades, escalabilidad funcional y claridad en la comunicacion entre componentes. Los clientes se mantienen enfocados en la presentacion, el Gateway en la entrada y coordinacion, los microservicios en la logica de negocio y la capa de persistencia en el almacenamiento y mensajeria del sistema.

Chapter 7. Diagrama de Despliegue Docker

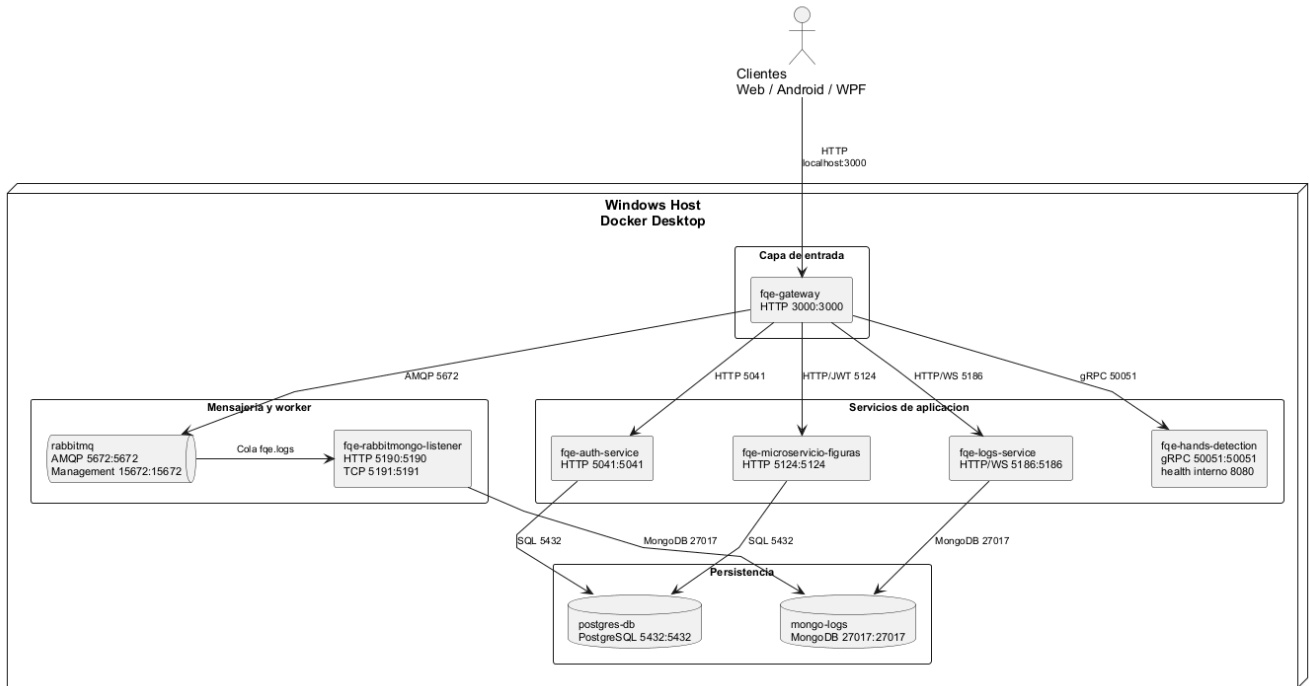
Esta vista muestra el despliegue local del sistema usando Docker Desktop en Windows. Los contenedores se ejecutan de forma independiente y publican puertos hacia la maquina host. Como los servicios se levantan desde distintos `docker-compose.yml`, la comunicacion entre contenedores usa `host.docker.internal` y los puertos publicados en Windows.

7.1. Puertos Publicados por Contenedor

Contenedor	Puerto en Windows	Puerto interno	Funcion
fqe-gateway	3000	3000	Punto de entrada HTTP para clientes web, Android y WPF.
fqe-auth-service	5041	5041	Servicio de autenticacion y emision de JWT.
fqe-microservicio-figuras	5124	5124	Servicio principal de datos educativos.
fqe-logs-service	5186	5186	Consulta de logs por HTTP y WebSocket.
fqe-rabbitmongo-listener	5190, 5191	5190, 5191	Worker de RabbitMQ a MongoDB y endpoint tecnico de salud.
fqe-hands-detection	50051	50051	Servicio gRPC de deteccion de manos.
postgres-db	5432	5432	Base de datos relacional PostgreSQL.
mongo-logs	27017	27017	Base de datos documental para logs.
rabbitmq	5672, 15672	5672, 15672	Mensajeria AMQP y consola de administracion RabbitMQ.

7.2. Diagrama de Despliegue

Diagrama de Despliegue Docker - Figuras Que Enseñan



7.3. Acceso a Servicios

La siguiente tabla resume como se accede a cada componente desde Windows y como se referencia desde otros contenedores Docker. En este despliegue, los contenedores se comunican usando `host.docker.internal` porque los servicios están publicados hacia el host.

Servicio o contenedor	Acceso desde Windows	Acceso desde Docker	Uso principal
fqe-gateway	http://localhost:3000	http://host.docker.internal:3000	Punto de entrada para clientes web, Android y WPF.
fqe-auth-service	http://localhost:5041	http://host.docker.internal:5041	Autenticación, registro y emisión de JWT.
fqe-microservicio-figuras	http://localhost:5124	http://host.docker.internal:5124	Datos educativos: estudiantes, tutores, sesiones, niveles y resultados.
fqe-logs-service	http://localhost:5186	http://host.docker.internal:5186	Consulta de logs y canal WebSocket de eventos.
fqe-rabbitmongo-listener	http://localhost:5190	http://host.docker.internal:5190	Monitoreo del worker que consume RabbitMQ y persiste eventos en MongoDB.
fqe-hands-detection	localhost:50051	host.docker.internal:50051	Servicio gRPC usado por el Gateway para detección de manos.

Servicio o contenedor	Acceso desde Windows	Acceso desde Docker	Uso principal
postgres-db	localhost:5432	host.docker.internal:5432	Base de datos relacional PostgreSQL.
mongo-logs	localhost:27017	host.docker.internal:27017	Base de datos documental para logs.
rabbitmq	amqp://localhost:5672 y http://localhost:15672	amqp://host.docker.internal:5672	Mensajería AMQP y consola de administración RabbitMQ.

Chapter 8. Persistencia de Datos

El sistema utiliza dos mecanismos principales de persistencia: una base de datos relacional en PostgreSQL y una base de datos no relacional en MongoDB. Ambas se ejecutan en contenedores Docker dentro del entorno local del proyecto y cumplen funciones diferentes dentro de la arquitectura.

PostgreSQL se utiliza para almacenar la informacion estructural y principal del dominio educativo, como usuarios, tutores, estudiantes, sesiones, niveles y resultados. MongoDB se utiliza para almacenar eventos y logs generados por los servicios, permitiendo conservar informacion operativa con una estructura mas flexible.

En el entorno revisado, PostgreSQL se encuentra montado en el contenedor Docker `postgres-db`, mientras que MongoDB se encuentra montado en el contenedor `mongo-logs`.

8.1. Base de Datos Relacional: PostgreSQL

La base de datos PostgreSQL corresponde a la persistencia principal del sistema. En ella se almacena la informacion que requiere integridad, relaciones claras entre entidades y consistencia en las operaciones.

Elemento	Descripcion
Motor	PostgreSQL
Contenedor Docker	<code>postgres-db</code>
Base de datos	<code>figurasqe</code>
Uso principal	Persistencia de usuarios, roles, estudiantes, tutores, sesiones, niveles y resultados.
Servicios que la utilizan	Servicio de autenticacion y microservicio principal de datos.

8.1.1. Funcion Dentro del Sistema

PostgreSQL se utiliza para guardar los datos centrales de la aplicacion. Su funcion principal es mantener informacion consistente y relacionada entre los distintos elementos del sistema.

Entre las entidades principales se encuentran:

- `admins`: usuarios administradores del sistema.
- `students`: estudiantes que utilizan la aplicacion.
- `tutors`: tutores, profesores o responsables asociados a estudiantes.
- `sessions`: sesiones de uso registradas por estudiante.
- `levels`: niveles o actividades disponibles.
- `level_results`: resultados obtenidos por los estudiantes en cada nivel.

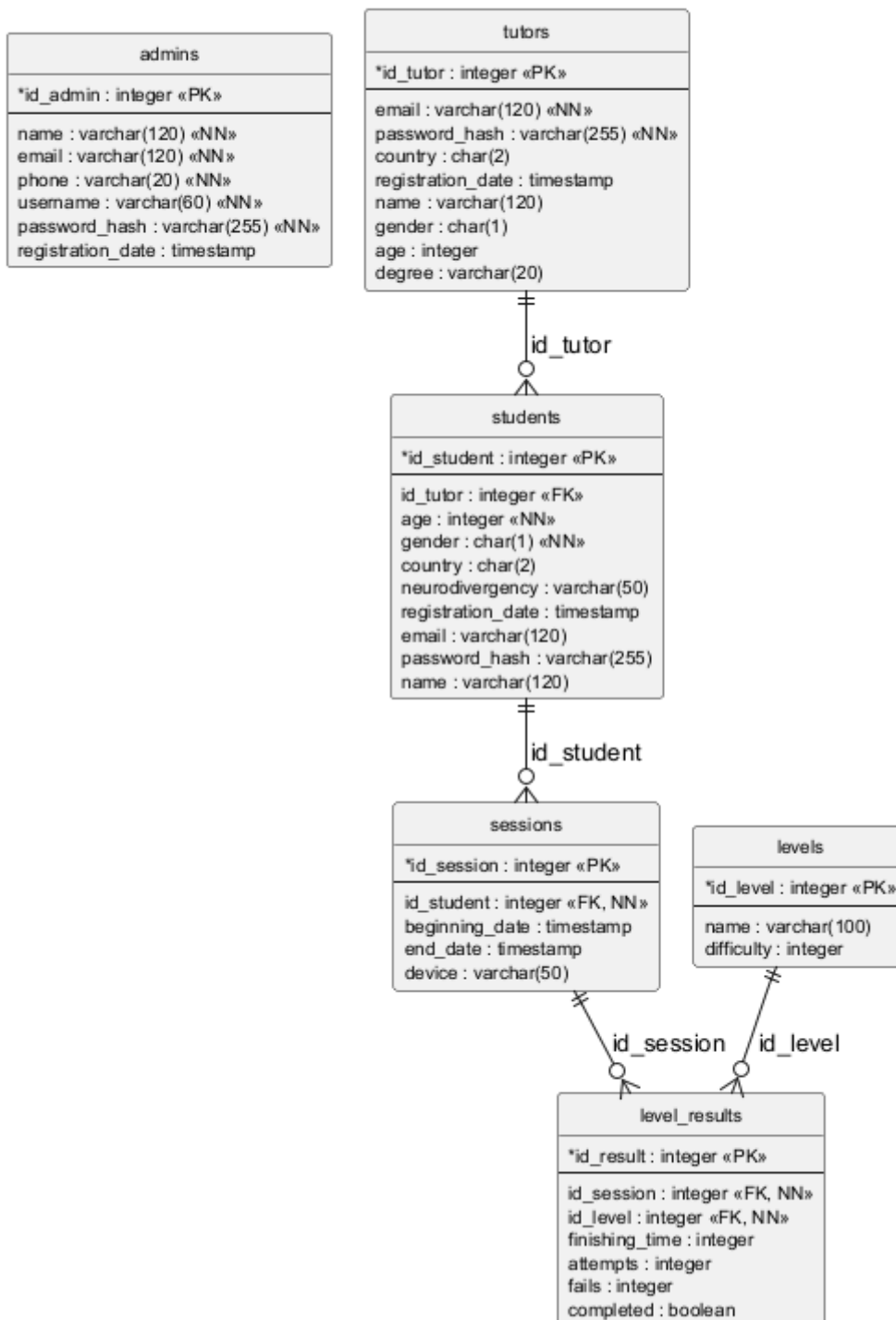
Estas entidades mantienen relaciones entre si. Por ejemplo, un tutor puede estar asociado a varios

estudiantes; un estudiante puede tener varias sesiones; y una sesion puede tener varios resultados de niveles. Este tipo de estructura requiere un modelo relacional para representar llaves primarias, llaves foraneas, restricciones e integridad referencial.

8.1.2. Diagrama PlantUML de la Base de Datos PostgreSQL

El siguiente diagrama representa la estructura real de la base de datos **figurasqe** consultada desde PostgreSQL. Se muestran las tablas principales del dominio, sus tipos de datos, llaves primarias, llaves foraneas y relaciones.

Modelo Relacional de la Base de Datos PostgreSQL - figurasqe



Restricciones relevantes detectadas en PostgreSQL:

- `admins.email` y `admins.username` tienen indices unicos.
- `tutors.email` tiene indice unico.
- `students.id_tutor` es una llave foranea nullable; por lo tanto, un estudiante puede existir sin tutor asignado.
- `students.gender` acepta los valores M, F u 0.
- `tutors.gender` acepta los valores M, F u 0.
- `tutors.degree` acepta los valores `licenciatura`, `maestria`, `doctorado`, `postdoctorado` o `padre-madre`.
- Las tablas `tutors`, `students`, `sessions`, `levels` y `level_results` usan secuencias para generar sus identificadores.

8.1.3. Justificacion del Modelo Relacional

PostgreSQL es adecuado para la informacion principal del sistema porque los datos tienen estructura definida y relaciones importantes. Los estudiantes, tutores, sesiones, niveles y resultados no son elementos aislados: dependen unos de otros y deben mantener coherencia.

El modelo relacional permite:

- Definir llaves primarias para identificar cada registro.
- Usar llaves foraneas para representar relaciones entre entidades.
- Evitar inconsistencias, como resultados asociados a sesiones inexistentes.
- Consultar informacion relacionada, por ejemplo estudiantes de un tutor o resultados de una sesion.
- Mantener reglas de integridad sobre usuarios, correos, sesiones y resultados.

Por estas razones, PostgreSQL se utiliza para los datos estructurados y criticos del dominio educativo.

8.2. Base de Datos No Relacional: MongoDB

MongoDB se utiliza como base de datos documental para almacenar logs y eventos generados por los servicios. A diferencia de la informacion principal del sistema, los logs pueden variar en estructura, contener metadatos diferentes y crecer continuamente conforme los servicios generan actividad.

En el entorno local revisado, MongoDB se ejecuta en Docker dentro del contenedor `mongo-logs`. La base de datos utilizada es `logsdb` y la coleccion principal es `events`.

Elemento	Descripcion
Motor	MongoDB
Contenedor Docker	<code>mongo-logs</code>
Base de datos	<code>logsdb</code>
Coleccion principal	<code>events</code>

Elemento	Descripcion
Uso principal	Persistencia de eventos, logs, errores, actividad tecnica y metadatos de mensajeria.
Servicios que la utilizan	Logs Service y RabbitMQ Mongo Listener.

8.2.1. Funcion Dentro del Sistema

MongoDB se encarga de almacenar informacion operativa y eventos del sistema. Los servicios pueden generar mensajes relacionados con actividad, errores, rutas consultadas, tiempos de respuesta, entidades involucradas o acciones realizadas.

El listener de RabbitMQ consume mensajes de una cola configurada, intenta interpretarlos como JSON y los guarda en MongoDB. Si el mensaje no tiene formato JSON valido, lo conserva como texto crudo mediante el campo `rawMessage`. Tambien puede agregar metadatos de RabbitMQ, como `deliveryTag`, `exchange`, `routingKey` y `redelivered`.

El servicio de logs consulta esta informacion desde MongoDB y la expone mediante endpoints HTTP y WebSocket, permitiendo al cliente administrativo revisar logs recientes, filtrarlos y consultar eventos del sistema.

8.2.2. Ejemplo de Documento de Log

```
{
  "service": "gateway",
  "route": "/data/students",
  "statusCode": 200,
  "durationMs": 45,
  "timestamp": "2026-05-21T20:30:00.000Z",
  "type": "event",
  "entityType": "student",
  "entityId": 12,
  "action": "query_students",
  "receivedAtUtc": "2026-05-21T20:30:01.000Z",
  "rabbitMetadata": {
    "exchange": "fqe.logs",
    "routingKey": "logs.gateway",
    "redelivered": false
  }
}
```

8.2.3. Estructura Detectada en MongoDB

La coleccion `events` contiene documentos de logs con campos orientados a registrar actividad tecnica y operativa. La estructura detectada incluye:

- `_id`: identificador automatico del documento.
- `service`: nombre del servicio que genero el evento.

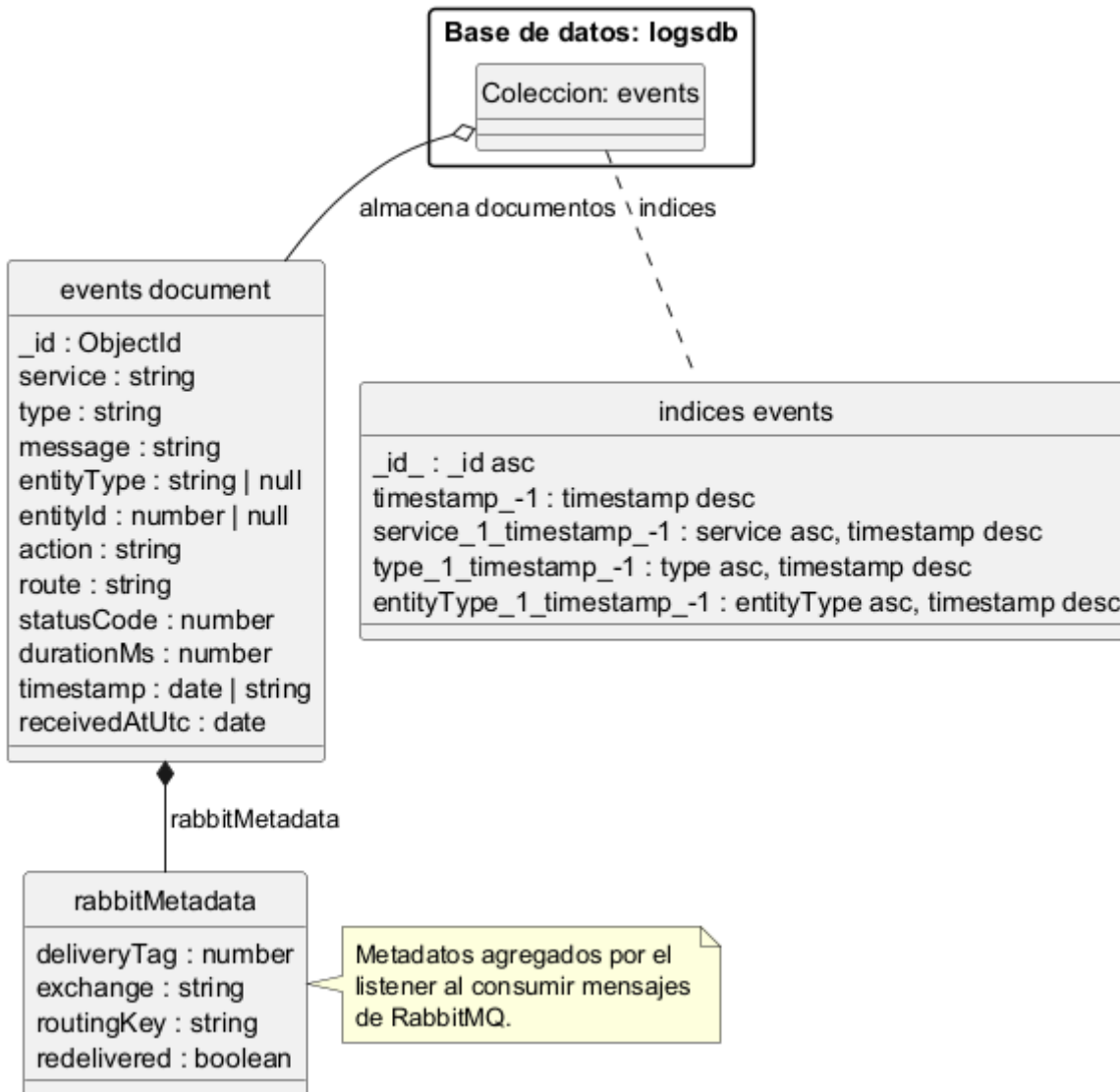
- **type**: tipo de evento, por ejemplo **success**, **error** o **event**.
- **message**: mensaje descriptivo del evento.
- **entityType**: tipo de entidad relacionada, por ejemplo **student**, **tutor**, **admin** o **null**.
- **entityId**: identificador de la entidad relacionada, cuando aplica.
- **action**: accion o metodo asociado al evento, por ejemplo **GET**, **POST** o **PUT**.
- **route**: ruta HTTP relacionada con el evento.
- **statusCode**: codigo HTTP de respuesta.
- **durationMs**: duracion de la solicitud en milisegundos.
- **timestamp**: fecha y hora del evento.
- **receivedAtUtc**: fecha y hora en que el evento fue recibido y almacenado.
- **rabbitMetadata**: objeto con metadatos de RabbitMQ.

La coleccion tambien cuenta con indices para acelerar consultas por fecha y filtros frecuentes:

- **id**
- **timestamp_-1**
- **service_1_timestamp_-1**
- **type_1_timestamp_-1**
- **entityType_1_timestamp_-1**

8.2.4. Diagrama PlantUML de MongoDB

Estructura de la Base de Datos MongoDB - logsdb



8.2.5. Justificacion del Modelo No Relacional

MongoDB es adecuado para logs y eventos porque esta informacion no siempre tiene una estructura fija. Un log puede incluir ruta, codigo de estado y duracion; otro puede incluir metadatos de RabbitMQ; otro puede representar un error; y otro puede llegar como mensaje crudo.

El modelo documental permite:

- Guardar eventos con campos variables sin modificar un esquema rigido.
- Almacenar documentos JSON de forma natural.
- Registrar metadatos adicionales conforme evolucionan los servicios.
- Consultar rapidamente eventos recientes y filtrarlos por servicio, tipo, ruta o accion.
- Separar la informacion operativa de los datos principales del dominio educativo.

Por estas razones, MongoDB se utiliza para informacion flexible, tecnica y orientada a eventos, mientras que PostgreSQL conserva los datos estructurados y relacionales del sistema.

Chapter 9. RabbitMQ

RabbitMQ se utiliza en el proyecto como mecanismo de mensajería para transportar eventos importantes generados por el sistema. Su uso principal actualmente está relacionado con el registro de logs: el gateway publica eventos en una cola y un servicio listener los consume para almacenarlos en MongoDB.

9.1. Montaje en Docker

RabbitMQ se ejecuta en un contenedor Docker llamado `rabbitmq`, usando la imagen `rabbitmq:3-management`.

Elemento	Valor
Contenedor	<code>rabbitmq</code>
Imagen	<code>rabbitmq:3-management</code>
Puerto AMQP	<code>5672</code>
Puerto de administración web	<code>15672</code>
Virtual host	<code>/</code>
Usuario por defecto	<code>guest</code>

El puerto `5672` se utiliza para la comunicación AMQP entre servicios. El puerto `15672` permite acceder a la consola web de administración de RabbitMQ.

9.2. Cola Principal del Proyecto

La cola utilizada por el flujo de logs del proyecto es:

```
fqe.logs
```

Esta cola se encuentra declarada como durable, por lo que RabbitMQ conserva su definición aunque el contenedor o el broker se reinicien. En el estado revisado, la cola existe dentro del vhost `/` y no tiene consumidores conectados ni mensajes pendientes.

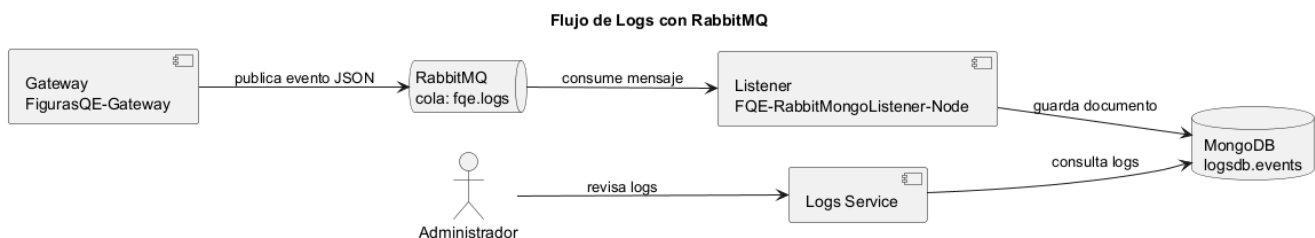
Propiedad	Valor
Nombre	<code>fqe.logs</code>
Durable	<code>true</code>
Auto delete	<code>false</code>
Exchange utilizado	Exchange directo por defecto
Routing key	<code>fqe.logs</code>
Uso	Transporte de eventos y logs hacia MongoDB

9.3. Flujo General

El flujo actual de RabbitMQ funciona de la siguiente forma:

1. El gateway recibe solicitudes HTTP de los clientes.
2. Un middleware identifica solicitudes importantes, como errores o acciones de escritura.
3. El gateway construye un evento de log en formato JSON.
4. El evento se publica en la cola `fqe.logs`.
5. El listener `FQE-RabbitMongoListener-Node` consume mensajes desde esa cola.
6. El listener interpreta el mensaje y agrega metadatos de RabbitMQ.
7. El evento se guarda como documento en MongoDB, dentro de `logsdb.events`.
8. El servicio de logs permite consultar esos documentos desde el cliente administrativo.

9.4. Diagrama del Flujo de Mensajería



9.5. Productor de Mensajes

El productor principal de mensajes es el gateway. En el código, el archivo encargado de publicar eventos es:

```
FigurasQE-Gateway/src/services/rabbitLogPublisher.js
```

El gateway utiliza la variable `RABBITMQ_URL` para conectarse al broker y `RABBITMQ_QUEUE` para determinar la cola destino. Si no se definen variables de entorno, utiliza valores por defecto:

Variable	Valor por defecto
<code>RABBITMQ_URL</code>	<code>amqp://guest:guest@localhost:5672</code>
<code>RABBITMQ_QUEUE</code>	<code>fqe.logs</code>

Antes de publicar, el gateway declara la cola mediante `assertQueue` con la opción `durable: true`. Los mensajes se publican con:

- `contentType: application/json`
- `persistent: true`

Esto indica que el mensaje contiene JSON y que RabbitMQ debe tratarlo como persistente.

9.6. Criterios para Publicar Logs

La publicacion de logs se controla desde el middleware:

```
FigurasQE-Gateway/src/middlewares/importantLogsMiddleware.js
```

El middleware registra eventos cuando:

- La ruta no esta excluida.
- El codigo de estado HTTP es mayor o igual al minimo configurado.
- El metodo HTTP pertenece a los metodos considerados importantes.

Valores por defecto:

Variable	Valor por defecto
IMPORTANT_LOG_ENABLED	true
IMPORTANT_LOG_METHODS	POST,PUT,PATCH,DELETE
IMPORTANT_LOG_STATUS_MIN	400
IMPORTANT_LOG_EXCLUDE_PREFIXES	/health,/logs

Con esta configuracion, el sistema registra acciones de escritura exitosas y solicitudes con errores, evitando rutas de salud y consulta de logs.

9.7. Estructura del Mensaje

El mensaje publicado en RabbitMQ es un JSON con informacion de la solicitud procesada.

```
{
  "service": "FiguerasQueEnse-anBD",
  "type": "success",
  "entityType": "student",
  "entityId": 30,
  "action": "PUT",
  "route": "/data/students/30",
  "statusCode": 200,
  "durationMs": 17,
  "timestamp": "2026-05-21T08:10:01.456Z"
}
```

Campos principales:

- **service**: servicio relacionado con la solicitud.
- **type**: tipo de evento, por ejemplo **success**, **error** o **event**.
- **entityType**: entidad involucrada, por ejemplo **student**, **tutor** o **admin**.

- **entityId**: identificador de la entidad cuando puede resolverse.
- **action**: metodo HTTP asociado.
- **route**: ruta consultada.
- **statusCode**: codigo HTTP de respuesta.
- **durationMs**: tiempo de procesamiento en milisegundos.
- **timestamp**: fecha y hora del evento.

9.8. Consumidor de Mensajes

El consumidor de mensajes es el servicio:

FQE-RabbitMongoListener-Node

Este servicio se conecta a RabbitMQ usando las variables separadas de host, puerto, usuario, password y vhost. Tambien se conecta a MongoDB para guardar los eventos procesados.

Variable	Valor por defecto
RABBITMQ_HOST	localhost
RABBITMQ_PORT	5672
RABBITMQ_USERNAME	guest
RABBITMQ_PASSWORD	guest
RABBITMQ_VHOST	/
RABBITMQ_QUEUE	fqe.logs
RABBITMQ_PREFETCH	20
RABBITMQ_AUTO_DECLARE	true
RABBITMQ_DURABLE_QUEUE	true
RECONNECT_DELAY_MS	5000

El listener declara la cola si **RABBITMQ_AUTO_DECLARE** esta habilitado. Tambien configura **prefetch** para limitar la cantidad de mensajes que puede recibir sin confirmar al mismo tiempo.

9.9. Confirmacion y Reintentos

El listener consume mensajes con confirmacion manual:

- Si el mensaje se guarda correctamente en MongoDB, el listener ejecuta **ack**.
- Si ocurre un error al procesar o guardar el mensaje, ejecuta **nack** con requeue habilitado.

Esto significa que un mensaje fallido vuelve a la cola para ser procesado nuevamente.

9.10. Metadatos Guardados

Cuando el listener recibe un mensaje, agrega metadatos de RabbitMQ antes de guardarlo en MongoDB:

```
{
  "rabbitMetadata": {
    "deliveryTag": 10,
    "exchange": "",
    "routingKey": "fqe.logs",
    "redelivered": false
  }
}
```

Estos metadatos ayudan a identificar como llego el mensaje al consumidor y si fue reenviado por RabbitMQ.

9.11. Cola Utilizada por el Proyecto

Aunque el broker pueda contener otras colas por pruebas o proyectos externos, la documentacion de Figuras Que Ensenan considera unicamente la cola utilizada por este sistema:

```
fqe.logs
```

Esta cola es usada para transportar eventos generados por el gateway y procesados posteriormente por el listener que guarda la informacion en MongoDB.

9.12. Consideraciones

- La cola `fqe.logs` es la unica cola documentada como parte del flujo de RabbitMQ de este proyecto.
- El gateway publica directamente en la cola usando el exchange por defecto.
- El listener consume desde la misma cola y guarda los mensajes en MongoDB.
- La configuracion usa mensajes persistentes y cola durable.
- El sistema depende de que RabbitMQ este disponible en el puerto `5672`.
- La consola web de RabbitMQ puede consultarse desde el puerto `15672`.
- Si el listener no esta activo, los mensajes pueden acumularse en la cola hasta que vuelva a consumirlos.

Chapter 10. JWT, Autenticacion y Autorizacion por Roles

El sistema utiliza JSON Web Token (JWT) para identificar al usuario autenticado y transportar su rol entre los clientes, el API Gateway y los microservicios. El token es generado por el microservicio de autenticacion y despues se envia en las solicitudes protegidas usando el encabezado **Authorization: Bearer**.

10.1. Generacion del JWT en Authentication Service

La generacion del token ocurre en el servicio de autenticacion. Cuando un usuario inicia sesion o se registra, el sistema consulta PostgreSQL para localizar la cuenta correspondiente y validar las credenciales. Las contraseñas almacenadas se comparan usando **BCrypt**.

Flujo	Descripcion
Estudiante o tutor	El cliente llama a POST /auth/login o POST /auth/register . El servicio busca al usuario en las tablas de estudiantes o tutores y genera un token con el rol student o tutor .
Administrador	El cliente administrativo llama a POST /auth/admin/login . El servicio busca al usuario en la tabla de administradores y genera un token con el rol admin .
Firma del token	El token se firma con HMAC SHA-256 usando el secreto configurado en Jwt:Secret .
Respuesta	La respuesta incluye el token y el role . En el flujo administrativo tambien se devuelve la informacion basica del administrador.

10.2. Informacion Incluida en el Token

El JWT contiene los datos necesarios para reconocer al usuario y su rol dentro del sistema.

Claim	Uso
sub	Identificador del usuario autenticado. Corresponde al id del estudiante, tutor o administrador.
email	Correo electronico de la cuenta autenticada.
role	Rol funcional del usuario. Los valores usados por el sistema son student , tutor y admin .
iss	Emisor del token, configurado como parte de JWT.
aud	Audiencia esperada del token.
iat	Fecha de emision del token.
exp	Fecha de expiracion del token.

10.3. Uso del JWT en los Clientes

Despues de recibir el token, cada cliente lo conserva y lo envia al Gateway cuando necesita consumir rutas protegidas.

Cliente	Uso del token
Cliente web	Lee el JWT despues del login, extrae role y sub , crea una sesion por cookie y guarda el token como claim jwt_token . Las paginas protegidas usan autorizacion por rol, por ejemplo student o tutor .
Cliente Android	Guarda el token localmente, decodifica el payload para leer role , email y sub , y envia solicitudes protegidas con Authorization: Bearer <token> .
Cliente WPF administrativo	Valida que la respuesta del login tenga role = admin , conserva el token en la sesion administrativa y lo envia como Bearer para consultar perfil, estadisticas, logs y monitoreo.

10.4. Papel del API Gateway

El API Gateway funciona como punto de entrada para los clientes. En las rutas protegidas, el Gateway conserva el encabezado **Authorization** recibido y lo reenvia hacia los servicios internos que requieren autenticacion.

```
Authorization: Bearer <jwt>
```

De esta forma, los clientes no necesitan conocer directamente la ubicacion de los microservicios internos, pero el token sigue llegando al servicio responsable de validar acceso.

10.5. Validacion y Autorizacion en el Microservicio de Datos

El microservicio principal de datos valida el JWT con **JwtBearer**. La validacion comprueba:

- Firma del token.
- Emisor configurado.
- Audiencia configurada.
- Expiracion del token.
- Existencia de un usuario autenticado para las rutas protegidas.

Ademas, el microservicio define politicas de autorizacion por rol.

Politica	Rol permitido
AdminOnly	admin
StudentOnly	student

Politica	Rol permitido
TutorOnly	tutor
StudentOrTutor	student o tutor

El sistema tambien usa el helper de claims para leer el identificador y el rol del usuario desde el token. Para el identificador se usa principalmente `sub`; para el rol se usa `role`. Con esa informacion los endpoints aplican reglas adicionales, por ejemplo:

- Un estudiante solo puede consultar informacion asociada a su propio identificador.
- Un tutor solo puede consultar estudiantes asociados a su cuenta.
- Un administrador puede consultar informacion administrativa, estadísticas, logs y monitoreo.

10.6. Determinacion del Rol

La determinacion de si el usuario es estudiante, tutor o administrador depende del claim `role` generado por Authentication Service.

Valor de <code>role</code>	Tipo de usuario	Uso dentro del sistema
<code>student</code>	Estudiante	Accede a su panel, perfil, tutor asignado, sesiones, resultados y actividades educativas.
<code>tutor</code>	Profesor, tutor o padre	Consulta su perfil, estudiantes asignados, sesiones y resultados de sus estudiantes.
<code>admin</code>	Administrador	Accede al cliente WPF administrativo, estadísticas, logs, monitoreo y gestion administrativa.

En resumen, Authentication Service es responsable de emitir el token con identidad y rol; los clientes lo conservan; el Gateway lo transporta; y los microservicios lo validan para permitir o rechazar operaciones segun el rol del usuario.

Chapter 11. Pruebas Automatizadas con Newman

Para validar el comportamiento de los servicios backend se preparo una bateria de pruebas automatizadas con Newman. Estas pruebas se ejecutaron sobre las colecciones del proyecto y se agruparon en una corrida unica que genera evidencia consolidada en formato JSON, JUnit y resumen general.

Suite	Cobertura principal	Resultado
Auth via Gateway	Login de estudiante, tutor y administrador; validaciones con body vacio, valores nulos, credenciales invalidas y registros exitosos.	Aprobada
Logs Service y Rabbit Listener	Health checks, respuestas 404 y 405, filtros de consulta y validacion end-to-end de persistencia de eventos en MongoDB.	Aprobada
Gateway	Health local, health de upstreams, documento OpenAPI y comportamiento base de los proxys hacia auth, logs, data y hands.	Aprobada
Data Service	Health, dashboard, admins, tutors, students, sessions, levels y level-results, incluyendo casos felices, validaciones 400 y restricciones 401/403.	Aprobada

La corrida consolidada utilizada como evidencia fue ejecutada el 26 de mayo de 2026 y produjo una carpeta de reportes con resumen general y archivos individuales por suite.

	executed	failed
iterations	1	0
requests	69	0
test-scripts	69	0
prerequest-scripts	69	0
assertions	114	0
total run duration: 8s		
total data received: 50.65kB (approx)		
average response time: 33ms [min: 2ms, max: 320ms, s.d.: 60ms]		

Figure 8. Resumen general de la corrida de Newman

```

→ Register - valores null
POST http://localhost:3000/auth/register [400 Bad Request, 496B, 24ms]
✓ Regresa 400 para nulls

→ Register - estudiante nuevo
POST http://localhost:3000/auth/register [200 OK, 610B, 280ms]
✓ Registra estudiante con 200
✓ Entrega token y role student

→ Login - estudiante recién registrado
POST http://localhost:3000/auth/login [200 OK, 610B, 196ms]
✓ Login del nuevo estudiante funciona
✓ Role sigue siendo student

→ Register - tutor nuevo
POST http://localhost:3000/auth/register [200 OK, 602B, 281ms]
✓ Registra tutor con 200
✓ Entrega token y role tutor

→ Login - tutor recién registrado
POST http://localhost:3000/auth/login [200 OK, 602B, 222ms]
✓ Login del nuevo tutor funciona
✓ Role sigue siendo tutor

```

	executed	failed
iterations	1	0
requests	13	0
test-scripts	13	0
prerequest-scripts	13	0
assertions	25	0
total run duration: 3.2s		
total data received: 3.59kB (approx)		
average response time: 148ms [min: 18ms, max: 281ms, s.d.: 95ms]		

Figure 9. Evidencia de aprobacion de Auth via Gateway

```

→ Gateway trigger - register unique student
POST http://localhost:3000/auth/register [200 OK, 604B, 298ms]
✓ Registro vía gateway exitoso
✓ Entrega token y role student

→ Verify listener captured auth register event
GET http://localhost:5186/api/logs?service=auth-service&route=/auth/register&limit=10 [200 OK, 2.48kB, 28ms]
✓ Consulta de logs responde 200
✓ El listener persiste el evento en Mongo y LogsService lo expone

```

	executed	failed
iterations	1	0
requests	9	0
test-scripts	9	0
prerequest-scripts	9	0
assertions	20	0
total run duration: 1334ms		
total data received: 4.64kB (approx)		
average response time: 51ms [min: 3ms, max: 298ms, s.d.: 88ms]		

Figure 10. Evidencia de aprobacion de Logs Service y Rabbit Listener


```

→ Gateway Data Proxy - students sin token
GET http://localhost:3000/data/students [401 Unauthorized, 309B, 98ms]
✓ Regresa 401 sin token

→ Gateway Logs Proxy - listado
GET http://localhost:3000/logs/api/logs?limit=2 [200 OK, 727B, 43ms]
✓ Regresa 200
✓ Proxy logs entrega arreglo

→ Gateway Logs Proxy - filtro por servicio
GET http://localhost:3000/logs/api/logs/service/auth-service?limit=3 [200 OK, 943B, 19ms]
✓ Regresa 200
✓ Entrega arreglo filtrado
✓ Todos pertenecen a auth-service

→ Gateway Hands Proxy - sin imagen
POST http://localhost:3000/hands [400 Bad Request, 305B, 5ms]
✓ Regresa 400 cuando falta image
✓ Mensaje esperado

```

	executed	failed
iterations	1	0
requests	11	0
test-scripts	11	0
prerequest-scripts	0	0
assertions	21	0
total run duration: 1621ms		
total data received: 13.3kB (approx)		
average response time: 52ms [min: 3ms, max: 220ms, s.d.: 60ms]		

Figure 11. Evidencia de aprobacion del Gateway

```

→ Level Results - obtener temporal
GET http://localhost:5124/level-results/74 [200 OK, 399B, 23ms]
✓ Level result temporal regresa 200
✓ Level result temporal correcto

→ Level Results - actualizar temporal
PUT http://localhost:5124/level-results/74 [200 OK, 399B, 58ms]
✓ Actualizar level result regresa 200
✓ Level result temporal actualizado

→ Level Results - eliminar temporal
DELETE http://localhost:5124/level-results/74 [200 OK, 92B, 64ms]
✓ Eliminar level result regresa 200

→ Sessions - eliminar temporal
DELETE http://localhost:5124/sessions/40 [200 OK, 92B, 25ms]
✓ Eliminar session regresa 200

→ Students - eliminar temporal
DELETE http://localhost:5124/students/59 [200 OK, 92B, 26ms]
✓ Eliminar student temporal regresa 200

→ Levels - eliminar temporal
DELETE http://localhost:5124/levels/16 [200 OK, 92B, 17ms]
✓ Eliminar level regresa 200

```

	executed	failed
iterations	1	0
requests	69	0
test-scripts	69	0
prerequest-scripts	69	0
assertions	114	0
total run duration: 8.9s		
total data received: 50.65kB (approx)		

Figure 12. Evidencia de aprobacion del Data Service

En la corrida consolidada se obtuvieron los siguientes resultados:

- **auth-gateway**: 13 solicitudes y 25 verificaciones, sin fallos.
- **logs-listener**: 9 solicitudes y 20 verificaciones, sin fallos.
- **gateway**: 11 solicitudes y 21 verificaciones, sin fallos.
- **data-service**: 69 solicitudes y 114 verificaciones, sin fallos.

Estas pruebas permiten demostrar no solo que los servicios responden, sino tambien que conservan codigos HTTP esperados, restricciones de autorizacion y persistencia correcta de informacion entre componentes.

Chapter 12. Pruebas de Estres con JMeter

Para complementar las pruebas funcionales con Newman, se ejecuto una prueba de estres sobre el gateway del sistema utilizando Apache JMeter. La ejecucion se realizo contra el gateway expuesto en <http://localhost:3000>, con una mezcla de solicitudes publicas y protegidas para aproximarse a un escenario de uso real.

Configuracion	Valor aplicado
Herramienta	Apache JMeter 5.6.3
Fecha de ejecucion	27 de mayo de 2026, 00:39 (America/Mexico_City)
Usuarios concurrentes	46
Ramp-up	15 segundos
Loops por usuario	10
Solicitudes esperadas por iteracion	5

La mezcla de trafico enviada al gateway incluyo los siguientes endpoints:

Endpoint	Objetivo
GET /health	Validar disponibilidad general del gateway.
GET /openapi.json	Verificar exposicion del contrato OpenAPI.
POST /auth/login	Medir el costo de autenticacion y generacion de JWT.
GET /data/students	Probar el paso hacia un endpoint protegido usando token JWT.
GET /logs/api/logs?limit=2	Validar el enrutamiento hacia el servicio de logs.

Los resultados consolidados de la corrida fueron los siguientes:

Metrica	Resultado
Solicitudes totales	2300
Errores	0
Porcentaje de error	0%
Throughput promedio	70.9 solicitudes por segundo
Tiempo medio de respuesta total	53.7 ms
Tiempo maximo observado	274 ms

De manera puntual, los tiempos medios por endpoint fueron:

Endpoint	Solicitudes	Tiempo medio
GET /health	460	6.1 ms
GET /openapi.json	460	6.2 ms

Endpoint	Solicitudes	Tiempo medio
POST /auth/login	460	209.0 ms
GET /data/students	460	23.1 ms
GET /logs/api/logs?limit=2	460	24.1 ms

La evidencia generada por JMeter quedo almacenada en una carpeta de resultados con archivo .jtl, bitacora de ejecucion y reporte HTML. En esta corrida, el reporte fue generado en C:\fig\jmeter-results\gateway-stress-20260527-003925\html-report\index.html.